

Virtual Context Memory

An Evidence-Carrying, Planner-Guided Context Compiler for Long-Horizon Language-Model Agents

Corben Sorenson

Version 1.0 - June 2026

Abstract

Long-horizon language-model agents are constrained not only by finite context windows, but by the absence of a complete contract for durable state. Retrieval, persistent memory, semantic paging, compression, predictive serving, and key-value-cache reuse each solve part of the problem; their composition can still lose identity, authority, evidence, consistency, privacy, or timing. This paper proposes **Virtual Context Memory (VCM)**, an inter-layer architecture that treats the model’s attention window as a compiled working set over a larger addressable memory space. Interaction history, task state, decisions, evidence, preferences, procedures, corrections, and rejected branches become typed semantic pages in a versioned context ledger. Stable addresses and mountable roots decouple logical identity from physical residency, while task-relative views range from routing capsules and structured syntheses to exact excerpts, raw sources, and compatible runtime caches. Every derived view carries a machine-auditable certificate declaring source bindings, omissions, authority ceilings, validity, and permitted uses. A Context Memory Management Unit predicts demand from an explicit plan, stages pages outside model-visible context, handles semantic faults, and invalidates stale descendants. A context compiler protects authorized policy, goals, constraints, corrections, commitments, procedures, and evidence obligations before allocating remaining capacity under token, latency, interference, privacy, and accelerator-memory budgets. Learned retention and prefetch policies may use only information observable when each decision is made; future-query labels can supervise offline learning but cannot leak into online evaluation. VCM does not claim lossless semantic compression or an infinite architectural window. Its falsifiable claim is that, for workloads with semantic and temporal locality, governed paging can enlarge effective working context across time while improving constraint survival, evidential fidelity, context-switch latency, and security. The paper specifies the architecture, invariants, failure responses, conformance profiles, and VCM-Bench evaluation needed to test that claim against both sophisticated and deliberately simple baselines.

Keywords— language-model agents; agent memory; context compiler; virtual memory; semantic paging; context compression; retrieval; KV cache; provenance; memory security.

Public preprint, Version 1.0. Conceptual architecture and research agenda. This paper specifies falsifiable mechanisms, invariants, and evaluation protocols; it does not report new empirical results.

1. Introduction

A language-model agent may interact with a person, codebase, institution, or environment for months while its underlying model can attend to only a bounded sequence of tokens at any one inference step. Enlarging the context window, retrieving older text, and recursively summarizing history all help, but they leave a deeper systems problem unresolved: **context is usually treated as text to append rather than state to manage.**

A durable interaction history is not a bag of interchangeable tokens. It contains hard constraints, current task state, decisions, corrections, evidence, user preferences, procedures, temporary assumptions, rejected designs, tool outputs, stale facts, quoted malicious instructions, and conversational residue. A flat transcript gives these objects similar structural status. Top- k retrieval privileges lexical or embedding similarity over behavioral importance. A rolling summary can collapse distinct source roles and silently lose negation, scope, or decision force. A large native window can hold more material, yet long-context benchmarks continue to show position sensitivity, distractor interference, and degradation from supplying more context than a task requires [1], [2], [3], [4]. Capacity and usable context are therefore related but not equivalent.

This paper begins from a systems premise:

The context window is a working cache, not the agent’s memory. Working context should be compiled, not merely accumulated.

We propose **Virtual Context Memory (VCM)**, an end-to-end contract for long-horizon agent memory. VCM exposes a large logical address space of typed semantic pages while keeping only a task-conditioned working set resident in active context. A project, corpus, relationship, case, or role can be mounted through a stable root address. The system then resolves descendant pages, predicts which representations will be needed, stages them before use, promotes only those that pass relevance and governance checks, and faults to stronger evidence when a summary is insufficient. Context switching becomes a change in mounted namespaces, task snapshot, and resident working set rather than a replay of the entire transcript.

The operating-systems analogy is productive but limited. Conventional virtual memory maps exact byte pages; semantic memory is meaning-dependent, variably sized, often lossy, and capable of changing an agent’s control flow. A semantic representation cannot be “decompressed” into omitted truth by generating plausible prose. Exact recovery is possible only for a retained lossless encoding or authoritative source. Otherwise the system must **materialize** a task-appropriate representation by resolving sources, selecting evidence, and synthesizing under an explicit use contract. Likewise, page replacement cannot depend on recency alone. It must consider constraints, failure-prevention value, dependency closure, contradiction, authority, temporal validity, privacy, poisoning risk, attention interference, materialization cost, and predicted reuse.

VCM is not the first work to compare LLM context with virtual memory. MemGPT introduced virtual-context management; recent systems have developed demand paging, learned page control, minimum-fidelity typed pages, provenance-linked summary-to-raw escalation, typed memory representations, lineage enforcement, programmable context compilation, workflow-aware prefetch, and model-runtime cache management [5], [6], [7], [8], [9], [10], [11], [12], [13]. A broader model-native architecture has also identified the need for a context compiler while emphasizing that semantic context management is intrinsically lossier than byte-addressed virtual memory [14].

The contribution here is therefore **contractual and compositional**, not a priority claim over each mechanism. VCM specifies the missing inter-layer interface that makes those mechanisms cohere: stable identity and mounts; task-relative representation contracts; evidence and authority preservation; protected context compilation; planner-guided non-model-visible staging; transactional snapshots; capability and taint enforcement; causal invalidation and deletion closure; and explicit mapping from semantic pages to disposable model-runtime state. In this sense, VCM is a proposed **Virtual Context ABI** between durable agent state, context-management policy, model execution, and the serving runtime.

1.1 Contributions

This paper contributes the following architecture and research specification:

1. **A Virtual Context ABI.** It defines stable addresses, immutable page versions, mountable roots, task snapshots, fault classes, representation contracts, and lifecycle operations that can be implemented in black-box, cooperative, or native model regimes.
2. **A task-relative representation graph.** It replaces a misleading linear “compression ladder” with multiple incomparable representations optimized for routing, broad semantic coverage, exactness, evidential support, privacy, or runtime speed. Selection is governed by an explicit use contract.
3. **Evidence-carrying derivation.** Every lossy or model-generated representation carries source bindings, atomic claims, scope, omissions, validity, contradictions, transformation lineage, verifier results, and permitted uses. Certificates establish integrity and lineage obligations; they do not pretend to prove natural-language truth.
4. **Authority non-escalation.** Compression, retrieval, summarization, and cache materialization cannot silently elevate quoted data into instruction, a source assertion into verified fact, or an inferred preference into a global behavioral rule.
5. **Protected context compilation.** The compiler authority-checks and admits a mandatory minimum set before optimizing discretionary pages. Untrusted text cannot self-pin. If the authorized set cannot fit, the system exposes infeasibility and changes model, budget, scope, or plan rather than silently dropping a constraint.
6. **Plan-conditioned semantic paging.** An explicit task-plan graph forecasts future page demand. Likely pages are asynchronously fetched and materialized into non-model-visible staging, calibrated by expected fault latency, materialization cost, privacy exposure, and interference risk.
7. **Transactional and governed memory semantics.** Immutable events, versioned materialized pages, copy-on-write task branches, contradiction and supersession edges, read-your-writes snapshots, capability-scoped mounts, taint, temporal validity, contestability, and deletion closure are part of memory management rather than optional filters.
8. **Cross-layer semantic/runtime coherence.** Semantic pages may be materialized as text, tokens, canonical ordered prefixes, latent adapters, or compatible KV objects, but model-specific forms remain disposable caches keyed by source version, exact prefix ancestry, position regime, model and adapter, tokenizer, role layout, policy, principal, redaction view, and task snapshot.
9. **A falsifiable evaluation program.** VCM-Bench tests mandatory-state survival, evidential fidelity, context-switch recovery, fault behavior, prefetch regret, attention interference, security, privacy, deletion closure, and cost across conversational and agentic workloads.

1.2 Scope and non-claims

VCM is a proposed architecture and research agenda. It does **not** claim that arbitrary semantic information can be compressed losslessly, that a short summary is sufficient for unknown future questions, that paging replaces tasks requiring simultaneous global attention, or that a finite-window model becomes equivalent to a model with unbounded native context. It does not

require access to a private chain-of-thought trace. Planner-guided prefetch operates on explicit, inspectable artifacts such as subgoals, dependency graphs, tool schedules, deadlines, and uncertainty estimates.

VCM also does not claim that role labels inside a prompt are a complete security boundary. Whenever possible, authority, tool permissions, data visibility, and cache isolation are enforced outside the language model by the governance kernel and runtime. Textual separation is a compatibility mechanism for black-box APIs, not the root of trust.

The strongest intended claim is narrower and testable: under workloads with semantic or temporal locality, a governed virtual-context system should let a finite-window agent use a substantially larger durable history with higher value per active token, lower context-switch and page-fault latency, better constraint and evidence preservation, and stronger security boundaries than transcript accumulation, similarity-only retrieval, or unverified recursive summaries. VCM may lose on short, one-shot, globally coupled, or low-reuse tasks; the evaluation is designed to reveal those regimes rather than hide them.

2. Problem Statement, System Model, and Requirements

2.1 Context as a managed resource

Let an agent operate over interaction steps $t=1,2,\dots$. At step t , it receives a task state T_t , environment state E_t , current user request U_t , policy set P_t , and access to a durable memory substrate M_t . The underlying model accepts at most B_t active tokens and may additionally use bounded host, accelerator, prefix, latent, or KV-cache resources.

The naïve objective is to select text that appears relevant. The actual objective is multi-resource and safety-constrained: preserve mandatory instructions; expose sufficient task state; include evidence needed for claims and tool actions; avoid stale, contradictory, poisoned, or unauthorized memory; respect privacy and purpose; maintain a coherent snapshot; and meet token and latency budgets without flooding attention with distractors. Context assembly is therefore a constrained compilation problem, not a similarity search.

We use **durable memory** for addressable state that can survive model calls and sessions; **working context** for the finite, role- and provenance-labeled representation supplied to one inference step or tightly coupled phase; and **runtime cache** for model-specific disposable state such as tokenized prefixes or KV blocks. Durable memory may be large while working context remains small. Conversely, a large context window can be badly managed memory.

2.2 System model

VCM assumes the following logical components:

- a base language model or model ensemble;
- an explicit task planner that emits inspectable subgoals, dependencies, branch probabilities, and deadlines;
- an immutable event substrate and versioned context ledger;
- indexes for semantic, lexical, symbolic, temporal, dependency, contradiction, and provenance retrieval;
- a Context Memory Management Unit (CMMU) that resolves addresses and manages staging, faults, and residency;
- a context compiler that produces model-ready working packets under protected-lane and resource constraints;
- a governance kernel that enforces capabilities, provenance roles, behavioral authority, privacy, freshness, taint, retention, and deletion;
- optional integration with tokenizers, attention masks, prefix caches, latent state, and KV-cache runtimes.

The architecture supports three deployment regimes. A **black-box regime** can only assemble text around an external model API and handle faults between calls. A **cooperative regime** controls structured prompts, tool boundaries, prefix caching, or model adapters. A **native regime** can expose page-aware attention, structured fault interrupts, context IDs, and hardware/runtime scheduling. The logical ABI is shared; latency, enforcement strength, and fault granularity differ.

2.3 Threat and fault model

The memory system must remain useful under benign error, distribution shift, and adversarial pressure. Threats include:

- malicious instructions embedded in webpages, files, tool outputs, or prior dialogue;
- direct or indirect attempts to write persistent behavioral instructions into memory;
- compromised or low-trust agents sharing memory;
- stale facts, superseded decisions, and ambiguous aliases;
- incorrect assistant-generated memories that become self-reinforcing;

- privacy leakage through retrieval, speculative prefetch, address enumeration, or access-pattern metadata;
- summary drift, lost negation, scope collapse, and unsupported reconstruction;
- unauthorized resurrection of sealed or purpose-limited pages;
- address substitution, rollback, signature failure, or alias poisoning;
- cache confusion across principals, models, tokenizers, policies, redaction views, or task snapshots;
- partial writes, write skew, inconsistent reads, and causal races during multi-agent execution;
- planner manipulation that induces wasteful or privacy-invasive prefetch;
- protected-lane stuffing in which untrusted content claims to be mandatory, urgent, or policy-like;
- self-reinforcing importance policies that permanently bury unfamiliar but critical memory;
- evaluation or policy leakage in which retention is scored using a future query, answer, dependency label, or page boundary unavailable when the decision was made;
- working-set overflow and thrashing when the task requires more simultaneous state than the active window can hold.

We assume a small trusted computing base comprising the governance kernel, cryptographic primitives, immutable-log integrity, capability validation, and system-level policy. The language model is not trusted to enforce access control, preserve source role after free-form prompt fusion, judge every attack correctly, or reconstruct omitted content faithfully.

2.4 Design requirements

A conforming VCM system should satisfy the following requirements.

R1 — Addressability. Every durable object has a stable logical address independent of physical storage, active representation, or current model.

R2 — Typed semantics. Policy, constraint, decision, correction, evidence, preference, procedure, tool output, and quoted untrusted text remain distinguishable through ingestion, derivation, retrieval, compilation, and writeback.

R3 — Evidence-linked derivation. Every derived representation retains a navigable lineage to authoritative sources, or explicitly declares that the source was not retained and narrows its permitted uses.

R4 — Declared loss and use contracts. Lossy derivation states what may have been omitted, the query/use families it was tested for, and the conditions that require escalation.

R5 — Authority non-escalation. No transformation, retrieval rank, repetition, or cache hit may grant content more evidential or behavioral authority than its sources and governing policy allow.

R6 — Dynamic residency. Any page may move directly from archival storage to staging or active context when demanded; physical tiers do not impose fixed semantic rings.

R7 — Predictive, non-model-visible staging. Explicit plans and dependency forecasts may hide fetch and materialization latency, but speculative pages remain outside model-visible generation state until promoted through relevance and governance gates. Staging may still consume resources or reveal access patterns, so those effects remain governed and measured.

R8 — Authorized mandatory preservation and visible infeasibility. Applicable policy, the current objective, authorized hard constraints, latest relevant corrections, unresolved commitments, required procedures, and evidence obligations receive protected capacity. Entry into this set is itself authority-checked: retrieved text cannot declare itself mandatory, pin itself, or manufacture an evidence obligation. If the authorized minimum cannot fit, the system reports an unsafe fit and changes budget, model, scope, or plan instead of dropping it silently.

R9 — Consistent views. A task reasons over a coherent named snapshot, observes its own committed writes, and does not silently combine incompatible versions or permission views.

R10 — Least privilege and purpose limitation. Reads, writes, mounts, promotions, sharing, prefetch, and tool-affecting interpretations are capability-checked and purpose-bounded.

R11 — Contestability. A user or auditor can inspect why a page was remembered, why it entered context, which sources support it, which transformations affected it, and how to correct, scope, seal, or delete it.

R12 — Cross-layer coherence. Text, embeddings, summaries, tokens, latent forms, prefixes, and KV pages are invalidated when semantically relevant source, model, tokenizer, role layout, policy, principal, permission, redaction, or task-snapshot keys change.

R13 — Policy/mechanism separation. Learned models may propose pages, scores, summaries, and predictions; deterministic mechanisms enforce hard invariants, capabilities, protected lanes, and cache isolation.

R14 — Graceful faulting. When identity, evidence, exactness, permission, freshness, consistency, or representation sufficiency is missing, the system faults explicitly, escalates to a valid representation, asks for clarification or permission, changes strategy, or declines to rely on the page.

R15 — Bounded interference. Context selection accounts not only for token cost but for attention pollution and behavioral interference caused by irrelevant or low-authority material.

R16 — Auditable lifecycle closure. Updates and deletion propagate to descendants, indexes, staging, replicas, and runtime caches with measurable completion and explicit residual limits such as backups or trained-model influence.

R17 — Observability-safe valuation. A retention, compression, or prefetch decision may use only state observable at that decision time. Future queries, answers, oracle dependencies, and benchmark annotations may provide offline supervision or upper bounds, but they cannot be exposed to the deployed policy or silently used in the primary evaluation.

3. Related Work and the Precise Novelty Boundary

VCM sits at the intersection of long-context modeling, retrieval, persistent agent memory, semantic paging, prompt compilation, model serving, provenance, and security. Individual mechanisms are increasingly well developed. The unresolved research question is whether they can be composed through a common contract without losing correctness or governance at their boundaries.

3.1 Long context is capacity, not governance

Long-context models can process far more tokens than early transformer systems, but nominal length does not imply reliable use. “Lost in the Middle” demonstrated position-dependent access to relevant information [1]. RULER and LongBench v2 evaluate more demanding long-context behaviors than simple needle retrieval [2], [3]. Performance can degrade even under perfect retrieval as excess supplied context burdens reasoning [4]. Native context growth remains valuable, especially for phases requiring simultaneous comparison, but it does not by itself provide typing, provenance, retention policy, contradiction handling, or lifecycle governance.

3.2 Retrieval and active retrieval

Retrieval-augmented generation externalizes documents and retrieves passages at inference time [15]. FLARE, Self-RAG, and IRCOT make retrieval conditional on generation or intermediate reasoning [16], [17], [18]. Predictive Prefetching for RAG forecasts when and what to retrieve so that external I/O can overlap generation [19]. These systems establish that context access can be dynamic and, in some settings, anticipatory.

VCM retains retrieval but separates it from residency. Predictive RAG operates over impending information needs during a generation; VCM additionally manages durable typed pages across task horizons, stages them outside model-visible state, and subjects promotion, retention, and reuse to snapshot, evidence, authority, privacy, and invalidation contracts. A retrieved object can remain staged, be promoted at a selected representation, be pinned across a plan phase, be mapped to a prefix/KV cache, or be denied because its role, freshness, purpose, or authority is unsuitable. It also retrieves by typed obligation—constraint, correction, procedure, rejection, evidence, or dependency—not only semantic similarity.

3.3 Persistent and hierarchical agent memory

Generative Agents combined observations, reflection, and planning in a persistent memory stream [20]. MemGPT, MemoryOS, and MemOS use operating-system-inspired memory abstractions [5], [21], [22]. Mem0, A-MEM, SimpleMem, and Agentic Memory address scalable storage, linked notes, lifelong memory, or learned memory operations [23], [24], [25], [26]. HiAgent, MemAgent, and ByteRover explore hierarchical working memory, learned recurrent memory, and LLM-curated context trees [27], [28], [29].

Recent systems sharpen the substrate itself. Contextual Memory Virtualisation represents accumulated session state as a versioned DAG with snapshot, branch, and structurally lossless trimming operations [30]. The View-oriented Conversation Compiler parses agent traces into a typed intermediate representation and emits lossless, user-facing, and adaptive views [31]. MemMachine keeps complete episodes as ground truth and expands retrieved nuclei with surrounding context rather than

relying primarily on lossy write-time extraction [32]. Portable Agent Memory defines a provenance-verified serialization and re-hydration protocol spanning episodic, semantic, procedural, working, and identity memory across heterogeneous agent runtimes [33]. These approaches support VCM’s insistence that compact views remain anchored to richer records and that durable memory should outlive a particular model or vendor. Portable Agent Memory focuses on interchange; VCM focuses on the live residency, compilation, fault, and coherence contract after memory is mounted.

Architectural complexity is not automatically an advantage. ENGRAM reports strong long-horizon results with three typed memory classes and straightforward dense retrieval, while the Deterministic Memory Framework removes model calls from memory preparation and uses source-linked deterministic scoring [34], [35]. They are important counterexamples to maximalist memory stacks: VCM is justified only if its additional contracts improve fidelity, latency, governance, or total cost beyond well-engineered simple systems.

Recent surveys likewise argue that modern agent memory cannot be captured by a simple short-term/long-term split and must be analyzed across representation, function, dynamics, evaluation, and trustworthiness [36]. These systems motivate durable, structured, and adaptive memory. VCM’s additional claim is that persistent memory requires a portable semantic/runtime contract: stable identity, representation eligibility, protected compilation, snapshot consistency, fault semantics, capability-scoped mounts, causal invalidation, and explicit runtime-cache coherence.

3.4 Semantic paging and harness-enforced residency

The Missing Memory Hierarchy develops demand paging for production context windows and reports the importance of working-set control and thrashing behavior [6]. Neural Paging formalizes a Context Paging Problem and learns a controller intended to approximate future-aware replacement under stated assumptions [7]. ClawVM is an especially close neighbor: it places typed pages, minimum-fidelity invariants, multi-resolution representations, lifecycle-aware writeback, deterministic prompt assembly, and an offline fault oracle in the agent harness [8].

RAMPART treats context assembly as an explicit programmable compile step over named, addressable in-RAM blocks, with promotion, gating, write, eviction, rollback, provenance tags, authorship controls, and position-sensitive placement [12]. It is a particularly close demonstration that compilation policy and structural placement matter independently of retrieval quality.

VCM accepts these advances rather than relabeling them. It extends the contract across boundaries that are individually outside or secondary to those works: source-backed multi-resolution paging beyond an in-RAM registry; task-relative evidence and authority contracts; planner-guided staging before demand faults; versioned multi-task and multi-agent snapshots; capability, taint, privacy, and deletion semantics; explicit infeasibility for mandatory-set overflow; and mapping from semantic identity to model-specific prefix/KV materializations.

3.5 Provenance-aware escalation, typed memory, and lineage

TierMem identifies the **write-before-query barrier**: a summary is created before the future query reveals which omitted detail matters. It uses provenance-linked summary and raw-log tiers with query-time sufficiency routing [9]. MemIR combats provenance-role collapse through typed grounded atoms and claim-centered projection [10]. MemLineage attaches cryptographic provenance and derivation lineage to memory and gates sensitive actions based on ancestry [11].

Structured execution records and multi-agent memory protocols expose adjacent requirements. The Agent Execution Record model represents intent, observations, inferences, versioned plans, evidence chains, and delegation authority as queryable operational artifacts [37]. The Mesh Memory Protocol emphasizes per-field admission and signal-level lineage so that agents can distinguish independent support from recycled echoes [38]. VCM can store either schema as typed event or semantic pages, but it does not claim that hidden chain-of-thought can be reconstructed from checkpoints; only explicitly captured operational state enters the ledger.

VCM generalizes these principles into a representation contract used by the compiler, fault handler, and governance kernel. A summary can guide routing without being eligible for quotation; a source assertion can support attribution without becoming externally verified fact; an untrusted ancestor can remain recallable while being barred from sensitive control flow. Lineage affects both epistemic use and execution authority.

3.6 Compression and active context management

LLMLingua, LongLLMLingua, gist tokens, and RECOMP reduce the amount of text supplied at inference [39], [40], [41], [42]. Active Context Compression lets an agent invoke compression as part of its own control loop [43]. These methods

demonstrate useful compression but also expose a central limitation: a compact form optimized for one use can be unsafe for another.

VCM therefore treats compression as creation of a certified derivative, not replacement of the source by default. Exact decode and semantic materialization are distinct operations. When a future use exceeds a representation’s contract, the correct behavior is a fault to evidence or an explicit inability to recover—not fluent reconstruction.

3.7 Runtime paging, prefix reuse, and predictive serving

PagedAttention, vAttention, SGLang’s RadixAttention, KVFlow, H2O, and StreamingLLM manage physical inference state, prefix reuse, or KV retention [44], [45], [46], [47], [48], [49]. SpeContext overlaps sparse-context/KV retrieval with model computation, while Pythia exploits multi-agent workflow predictability for serving decisions [13], [50]. PBKV predicts dynamic workflow invocations to guide conservative KV eviction and prefetch, and TokenCake uses application graphs and function-call events for predictive upload, offload, and critical-path reservation [51], [52]. Leyline exposes policy-directed KV edit primitives for agentic traces, and fail-closed ResidentClaim lowering distinguishes generic cache controls from a runtime’s accepted obligation to make a specific future materialization reusable [53], [54].

These systems manage bytes, tokens, prefixes, and KV blocks rather than durable semantic truth. VCM links the layers through strict derivation keys and accepted lifecycle claims: semantic pages are authoritative logical objects; runtime forms are disposable, model-versioned materializations. Planner demand can guide runtime prefetch, but a cache hit cannot establish freshness, authorization, provenance, or truth, and a priority or offload hint is not equivalent to a guaranteed future reuse contract.

3.8 Security, privacy, staleness, and personalization

Memory injection, experience poisoning, and memory-control-flow attacks show that persistent memory is a durable attack surface [55], [56], [57], [58]. A recent lifecycle survey organizes memory risk across write, store, retrieve, execute, share, and forget/rollback phases and argues for verifiable memory governance [59]. MemGate treats memory search itself as a trust boundary and gates semantically related but contextually unsafe memories before they reach the backbone model [60]. OP-Bench examines over-personalization, STALE tests invalid memories, and MemPrivacy studies privacy-preserving personalized memory [61], [62], [63].

VCM treats role, authority, capability, taint, purpose, temporal validity, and deletion as first-class memory metadata. Speculative prefetch is an access event. A summary cannot wash away untrusted ancestry. A scoped preference cannot silently become a global identity claim. Deletion targets the full derivation and cache closure, while acknowledging limits such as backups and already-trained model weights.

3.9 Broader model-native architecture

The Model-Native Computing Architecture surveys an emerging full stack and explicitly identifies context management, semantic locality, and a context compiler as system-level needs [14]. Its scope is a six-layer architectural vision. VCM is narrower and deeper: it specifies the context-memory layer’s page identity, representation contracts, fault model, compiler obligations, transaction semantics, governance, and conformance surface. The two views are complementary rather than competing priority claims.

3.10 Comparative position

Table 1 focuses on the nearest architectural neighbors. “Core” denotes a central, explicit mechanism; “partial” denotes a related mechanism with narrower scope; “outside scope” does not imply a flaw.

Table 1. Primary emphasis of close architectural neighbors. The comparison is intentionally conservative and should be updated as these rapidly evolving systems mature.

System family	Typed / minimum-fidelity state	Query-time evidence escalation	Plan-guided non-visible staging	Snapshot / invalidation contract	Authority and capability enforcement	Semantic-to-runtime cache binding
MemGPT / memory OS systems	Partial	Partial	Limited	System-dependent	Emerging	Limited
CMV / VCC / MemMachine	Structured or source-preserving	Source replay or exact trace	Limited	Core in CMV; trace-centered	Limited	Outside primary scope

System family	Typed / minimum-fidelity state	Query-time evidence escalation	Plan-guided non-visible staging	Snapshot / invalidation contract	Authority and capability enforcement	Semantic-to-runtime cache binding
RAMPART	views Named addressable blocks and compiled placement	pointers No dedicated evidence escalation	No; in-RAM pre-compilation	in VCC Session-local rollback	Block ownership and authorship flags	Outside primary scope
Portable Agent Memory	Typed portable memory package	Source/provenance verification	Re-hydration rather than plan prefetch	Transfer manifest and Merkle-DAG	Capability-scoped transfer	Model-agnostic interchange
ENGRAM / DMF	Lightweight typed or deterministic records	Source-linked retrieval	No	Limited	Limited	Outside scope
Pichay / Neural Paging	Paging-centered	Limited	Learned or demand-centered	Limited	Limited	Limited
ClawVM	Core	Minimum-fidelity fallback	Limited	Lifecycle writeback; single-session emphasis	Harness policy	Outside primary scope
TierMem	Summary/raw tiers	Core	No	Writeback-centered	Limited	Outside scope
MemIR / MemLineage	Core provenance roles	Claim/lineage-centered	No	Lineage-centered	Core for sensitive use	Outside scope
Pythia / PBKV / TokenCake / SpeContext	Runtime objects	No	Core at serving/KV layer	Runtime keyed	Infrastructure-level	Core
Model-Native Architecture	Vision-level	Vision-level	Vision-level	Vision-level	Vision-level	Vision-level
VCM	Core, typed pages and protected minimum set	Core, use-contract faults	Core, staged from explicit plans	Core, task snapshots and causal invalidation	Core, authority non-escalation and capabilities	Core, disposable keyed materializations

The novelty claim is therefore precise: VCM does not introduce every row mechanism independently. It proposes a single auditable ABI that composes semantic paging, evidence sufficiency, protected compilation, prediction, transactions, governance, and runtime residency without allowing one layer’s optimization to violate another layer’s correctness contract.

4. Core Abstractions and Formal Model

4.1 Context cell, semantic page, and virtual address

A **context cell** is an atomic typed assertion or event unit: “decision D was accepted at time t ,” “source S supports claim C ,” “the user stated preference P within scope Q ,” or “branch B was rejected for reason R .” Cells are smaller than arbitrary text chunks so that source role, authority, temporal validity, and contradiction can be preserved independently.

A **semantic page** is the smallest independently addressable and governable bundle of cells intended to preserve a coherent unit of use. Page boundaries are semantic rather than fixed-byte. A page may encode a project checkpoint, decision, evidence bundle, procedure, preference capsule, code-symbol neighborhood, theorem dependency, or tool result. Co-access manifests may group pages into composite “huge pages,” but base page identities remain stable so that optimization does not destroy auditability.

A page has a logical address such as:

vcm://principal/namespace/object-id@version#view

The principal identifies the owning trust domain; the namespace identifies a mounted context space; the object identifier is stable; the version resolves to an immutable manifest; and the view selects a projection. Representation requirements are supplied separately as a use contract rather than encoded as an entitlement in the address. Human-readable aliases can point to immutable versions, but alias resolution is signed, rollback-protected, and audited.

A **context root** is a manifest naming a coherent address space—a project, corpus, relationship, case, codebase, or agent role. Mounting a root makes its namespace, routing capsules, capability view, and snapshot available. It does not load every descendant page. A single root address can therefore trigger a context switch while retaining fine-grained demand paging.

4.2 Page state

For page p_i , VCM records:

- logical identity, content hash, and immutable version;
- page type, provenance role, execution class, and maximum behavioral authority;
- authoritative source references and atomic claim bindings;
- dependency, contradiction, qualification, supersession, rejection, and causal relations;
- task-conditioned importance and risk vectors;
- temporal validity, freshness policy, and verification state;
- capabilities, purpose, retention, sharing, and prefetch policy;
- available representations and their certificates;
- physical residency and runtime-cache keys;
- access, promotion, denial, update, invalidation, and deletion history.

Let a_i be the address, τ_i the page type, x_i the authoritative payload, G_i the relation graph, V_i the temporal-validity state, K_i the capability policy, $A_i = (A_i^b, A_i^e, A_i^x)$ the behavioral, evidential, and action-authority ceilings, and $R_i = \{r_{i0}, \dots, r_{im}\}$ the available representations. Current action authorization is never inferred from memory alone; it is intersected with live capabilities at execution time.

4.3 Representation graph, not a linear ladder

A page can have several useful representations:

Table 2. Representations are specialized and only partially ordered. Exact excerpts can dominate a synthesis for quotation while providing less global coverage; a runtime cache can be faster while carrying no additional epistemic authority.

Representation	Optimized for	Principal limitation	Typical authority
Handle / manifest	Addressing, routing, dependency planning	Almost no substantive content	Index metadata
Routing capsule	Fast page selection and prefetch	Coarse, omission-heavy	Derived cue only
Structured synthesis	Broad semantic coverage at low token cost	Lossy and model-generated	Derived interpretation
Evidence bundle	Claim-to-source reasoning and careful synthesis	May omit unrelated source content	Source-bound derivative
Exact excerpts	Quotation, code, numbers, disputed wording	Narrow local coverage	Extractive source evidence
Raw event or document view	Audit and re-derivation	Expensive; may still contain untrusted assertions	Authoritative record, not necessarily true claim
Runtime materialization	Fast repeated inference	Model-, tokenizer-, prefix-, policy-, and snapshot-specific	Disposable cache only

Semantic pages separate identity, task-relative views, and residency

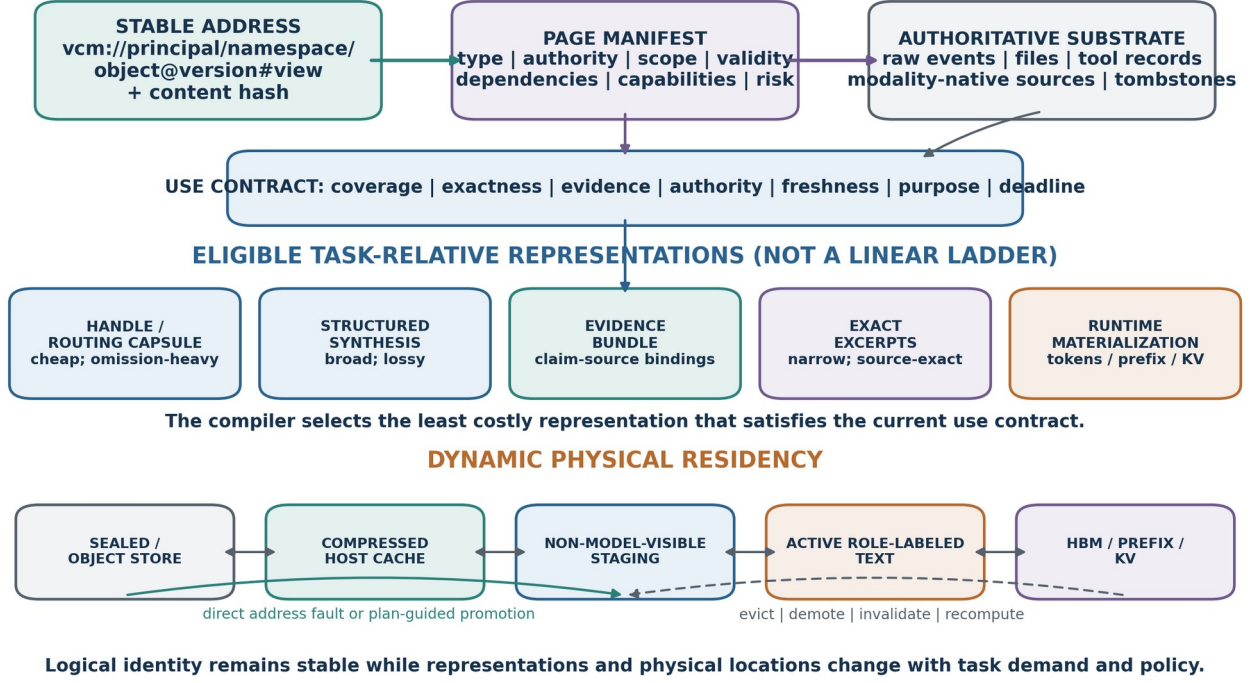


Figure 1. A semantic page has a stable identity but multiple task-relative representations and dynamic physical residencies.

For a requested operation, define a **use contract**

$$U = \{Q, c, e, a^{\min}, a^{\max}, f, \pi, d\},$$

where Q is the query or operation family, c the required semantic coverage, e the exactness and evidence obligation, $a^{\min}$ and $a^{\max}$ the required and permitted authority bounds, f the freshness condition, π the purpose and capability policy, and d the deadline. Authority is use-specific rather than scalar: at minimum, VCM distinguishes behavioral influence, evidential use, and action authorization. A representation certificate $\Gamma(r)$ declares corresponding guarantees, limits, cost, and lineage. A representation is eligible only if

$$\text{eligible}(r, U, s) = 1,$$

meaning that, under snapshot s , it satisfies the contract, is authorized, is not invalidated, and remains within every authority ceiling. The representation space is therefore a directed graph with task-relative dominance, not a universal scalar level.

4.4 Exact decoding versus semantic materialization

VCM distinguishes two operations that are often conflated:

1. **Exact decode** reconstructs an identical retained payload from a lossless encoding. Its correctness can be checked byte-for-byte.
2. **Semantic materialization** resolves sources and constructs a representation suitable for a declared use. It may be extractive or generative, but it cannot recover omitted information without consulting a source that still contains it.

A model-generated expansion of a summary is not decompression. It is a new derivative and must receive a new certificate. If the required fact is absent from both the resident representation and retained sources, VCM returns an unrecoverable-detail fault rather than authorizing plausible completion.

4.5 The future-query barrier

No strict lossy summary can be guaranteed sufficient for every unknown future question. The following elementary result makes the limitation explicit.

Proposition 1 — No universal strict summary. Let X be a finite set of possible source states and $g: X \rightarrow R$ a representation function with $|R| < |X|$. Then there exist $x \neq x'$ such that $g(x) = g(x')$. For any query language capable of distinguishing x from x' , there exists a query q for which no procedure that sees only $g(x)$ can answer correctly on both sources.

Proof. By the pigeonhole principle, two source states map to the same representation. Choose a query whose correct answer differs between those states. A decoder receiving the same representation must return the same output distribution for both, so it cannot be correct on both with certainty. $\square$

The proposition is intentionally modest, but it blocks an important category error: a summary written before the future query cannot promise arbitrary future sufficiency. This formalizes the write-before-query barrier identified by provenance-tiered memory [9].

Corollary 1 — Safe strict compression requires an escape hatch. Any system that stores a strict lossy representation for unknown future uses must do at least one of the following: retain or address a more informative source; restrict the representation to an explicitly bounded future-use family; or admit that some future requests will produce an unrecoverable-detail fault. A system that does none of the three can only hide information loss behind unsupported reconstruction.

VCM implements all three responses: it retains authoritative sources where policy permits, declares use-relative sufficiency, stores specialized representations, and faults to stronger evidence or explicit non-recoverability.

For anticipated query family Q , a derived representation r is (ϵ, Q) -sufficient when

$$E_{q \sim Q}[L(f(q, x), f(q, r))] \leq \epsilon,$$

where x is the authoritative source, f the downstream answer or decision procedure, and L a task-relevant loss. This is an empirical contract, not an ontological guarantee. Query distributions can shift, and high-stakes uses can require exact source escalation regardless of average loss.

4.6 Importance, risk, and anti-starvation

A scalar importance tag is too coarse. VCM uses a task-conditioned value vector

$$i_i(t) = \begin{bmatrix} \text{task relevance} & \text{future utility} & \text{constraint weight} \\ \text{decision weight} & \text{evidence value} & \text{failure prevention} \\ \text{stability} & \text{reuse probability} & \text{resurrection value} \end{bmatrix}$$

and a risk vector

$$r_i(t) = \begin{bmatrix} \text{source uncertainty} & \text{contradiction} & \text{staleness} \\ \text{privacy sensitivity} & \text{poisoning} & \text{role confusion} \\ \text{compression loss} & \text{attention interference} & \text{uncertainty} \end{bmatrix}.$$

Both are conditioned on task, user, time, policy, and snapshot. A dormant page can become critical when its project is mounted; a recent but irrelevant page can be evicted immediately. Importance is a current allocation estimate, not a permanent moral rank.

The architecture distinguishes three values that are often conflated:

1. **consolidation value**, estimated before an unknown future request and used for retention or encoding depth;
2. **request-conditioned relevance**, estimated after the current request is observable and used for retrieval and residency; and
3. **hindsight utility**, measured after an outcome and used for offline learning, audit, or counterfactual analysis.

A retention policy at time t must be measurable with respect to the information then available. If $O_{\leq t}$ is the observable history, an online decision has the form

$$d_t = \pi_\theta(F_t), F_t \in \sigma(O_{\leq t}),$$

not a function of an unseen future request, gold answer, or oracle dependency label. Future outcomes may train π_θ offline, but the evaluation must replay the policy from a decision-time snapshot. Recent work on multi-factor value and observability-safe retention shows why this matters: scoring “goal relevance” against the held-out question evaluates retrieval with answer-side information, not realistic forgetting [64], [65]. VCM therefore treats observability as part of the memory-policy interface, not merely an experimental detail.

Learned importance can also create a popularity loop: frequently retrieved pages receive more evidence of usefulness while buried pages never get reconsidered. VCM therefore reserves a bounded **resurfacing budget** for low-frequency but high-uncertainty pages, re-evaluates pages on mount, contradiction, and task changes, and never deletes a page solely because a learned policy predicts low use. Retention and deletion remain separate policy decisions.

4.7 Authority and evidence invariants

Let $A(r)$ denote the authority vector of representation r , $src(r)$ its source set, and $\leq$ the partial order of an implementation-defined authority lattice. A conforming transformation satisfies

$$A(r) \leq A_{\text{policy}} \sqcap \bigvee_{s \in src(r)} A(s),$$

and may be assigned a strictly lower ceiling when the transformation is inferential, lossy, stale, tainted, or insufficiently verified. This **authority non-escalation invariant** means that summarizing an external webpage cannot turn it into system policy; repeated retrieval cannot turn an inference into an explicit user preference; and caching a page cannot raise its authorization. Behavioral influence, evidential use, and action authority can have different orders, but enforcement occurs outside the model. Live tool authority is checked again at execution and is not inherited from a remembered page.

Evidence is separate from behavioral authority. An authoritative event record proves that a source said something, not that the statement is true. Representation certificates therefore distinguish source integrity, claim support, external verification, uncertainty, and execution permission.

4.8 Working-context compilation

At step t , let M_t be the **authorized** protected minimum set: applicable policy, the current request and objective, hard constraints issued by a principal with sufficient authority, latest relevant corrections, unresolved commitments, procedures required for imminent authorized actions, and evidence obligations for planned claims. Membership in M_t is a governance decision, not a label that page content can assign to itself. Let $c(r)$ denote active-token cost. Safe compilation first checks

$$\sum_{p_i \in M_t} c(r_i^{\min}(U_i)) \leq B_t,$$

where $r_i^{\min}(U_i)$ is the cheapest eligible representation for the mandatory use. If the inequality fails, the compiler returns **UNSAFE-FIT**. Valid responses include requesting a larger context budget, switching models, decomposing the task, postponing an action, narrowing scope, or asking the user. Silently omitting a mandatory page is not a legal optimization. Conversely, blindly pinning every instruction-like span would permit protected-lane denial of service. VCM therefore authenticates issuers, resolves precedence and scope, rate-limits low-trust writes, deduplicates equivalent obligations, and surfaces unresolved mandatory conflicts. Untrusted or merely retrieved content cannot self-pin.

After admitting M_t , the compiler chooses optional page representations W_t under token, latency, accelerator-memory, privacy-exposure, and attention-interference budgets:

$$W_t^i = \arg \max_{W \in F_t} E[Q(Y_t \mid W, T_t, E_t)] - \lambda C(W) - \mu R(W),$$

where F_t enforces eligibility, capability, snapshot, dependency, conflict, and authority constraints. Let $c(r)$ be token, latency, host-memory, accelerator-memory, privacy-exposure, and interference cost, and B_t the corresponding budget vector; every feasible packet satisfies

$$\sum_{r \in W} c(r) \leq B_t, M_t \subseteq W.$$

The objective is not generally additive. Pages can be complements through dependencies, substitutes through redundancy, or jointly harmful through interference. The problem is therefore a multiple-choice, dependency-constrained packing problem related to the deterministic prompt assembly used by ClawVM [8]. Exact solution is generally unnecessary and often intractable; implementations can use protected lanes, greedy marginal bids with interaction updates, constrained learned value estimates, branch-and-bound for small sets, or online admission control, provided the hard invariants remain external to the learned scorer.

An approximate optional bid is

$$b_{ik} = U_{ik}^{\text{task}} + U_{ik}^{\text{failure prevention}} + U_{ik}^{\text{evidence}} + U_{ik}^{\text{reuse}} - C_{ik}^{\text{tokens}} - C_{ik}^{\text{latency}} - C_{ik}^{\text{interference}} - C_{ik}^{\text{exposure}} - C_{ik}^{\text{drift}}.$$

Role labels inside the final prompt improve interpretability but are not trusted enforcement. Tool authorization, capability checks, sealed-page access, and execution-class restrictions remain in the runtime and governance kernel.

4.9 Effective working context

VCM does not enlarge the model’s architectural context. It enlarges the set of pages that can be made available, at sufficient fidelity and authority, before they are needed across a task horizon. Define

$$EWC(H; \delta, \epsilon) = \left\{ p_i : \Pr[L_i \leq \delta_i] \geq \rho \wedge \text{eligible}(r_i, u_i, s) = 1 \wedge r_i \text{ is } (\epsilon, Q_i)\text{-sufficient} \right\},$$

where L_i is materialization latency and δ_i the page deadline. A dormant archive does not count as effective context merely because it exists.

Useful companion metrics are:

- **Context value density:** expected useful and failure-preventing contribution per active token.
- **Effective context bandwidth:** eligible, source-linked context promoted per unit time.
- **Context interference:** quality loss attributable to included but non-causal or distracting context.
- **Context-switch recovery time:** time and model calls required to resume a mounted task to a specified correctness threshold.
- **Unsafe-fit rate:** fraction of episodes in which a system drops mandatory state instead of surfacing infeasibility.

5. Virtual Context Memory Architecture

VCM separates memory into three planes: a **data plane** that stores authoritative and derived state, a **control plane** that predicts and manages context demand, and a **governance plane** that decides whether and how memory may influence the agent. Their shared interface is the **Virtual Context ABI**: address resolution, mounting, use-contract evaluation, staging, promotion, faulting, snapshotting, commit, invalidation, and unmounting. Figure 2 shows the full architecture.

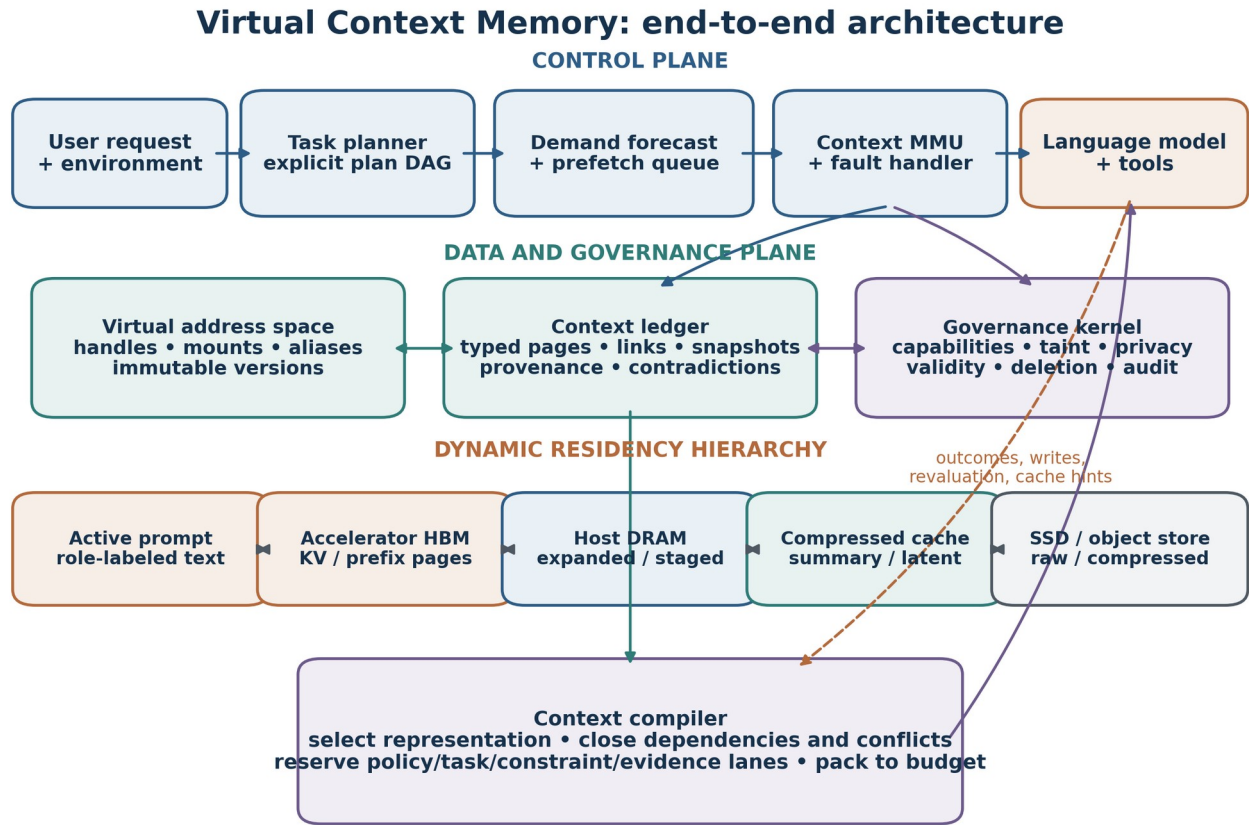


Figure 2. End-to-end Virtual Context Memory architecture.

5.1 Data plane: immutable events and the context ledger

The data plane contains two complementary substrates.

First, an **immutable event log** records user messages, agent outputs, tool calls, tool results, external documents, policy changes, memory edits, permissions, and deletion events. Events are content-addressed, timestamped, provenance-labeled, and ordered causally where possible. The event log is not dumped into prompts. It is the durable substrate from which memory can be re-derived, audited, or disputed.

Second, the **context ledger** contains typed, versioned pages derived from events and other pages. The ledger is a graph rather than a bag of embeddings. Its edges include:

- **supports**: evidence supports a claim or decision;
- **derived_from**: a representation or page descends from sources;
- **depends_on**: a task, theorem, plan, or procedure requires another page;
- **contradicts**: two pages make incompatible assertions within overlapping scope;
- **supersedes**: a newer decision or fact replaces an older one;
- **qualifies**: one page narrows the scope or certainty of another;
- **rejected_because**: a rejected branch records its rationale;
- **caused_by**: an action or observation has an explicit causal predecessor;
- **shared_with**: a capability-limited relation to another principal or agent;
- **invalidates**: an update makes a derived representation or cache stale.

Embeddings, keyword indexes, temporal indexes, symbolic fields, and graph neighborhoods are all materialized views over the ledger. No single index is the memory itself.

5.2 Page taxonomy

Page type determines default invariants, compression behavior, and retrieval channels. Table 3 gives a representative taxonomy.

Table 3. Representative semantic-page taxonomy and default lifecycle rules.

Page type	Required fields	Default residency / retention rule	Typical faults
Policy / hard constraint	issuer, precedence, scope, validity	Pinned while applicable; never merged into ordinary summary	version, conflict, capability
Task state	objective, status, dependencies, owner	Resident while task is active; checkpoint on switch	dependency, freshness
Decision	choice, alternatives, rationale, status	Preserve until superseded; retrieve by task and consequence	rationale, supersession
Correction	corrected claim, prior version, evidence	High failure-prevention value; invalidates descendants	contradiction, exactness
Evidence	source, claim map, trust, timestamp	Load to claim-required fidelity; retain provenance	exactness, freshness
Scoped preference	subject, scope, confidence, counterexamples	Retain only with bounded scope and review policy	scope, consent, validity
Procedure	preconditions, steps, permissions, version	Materialize exactly for execution; do not paraphrase critical steps silently	version, authorization
Episodic event	actors, time, event, uncertainty	Compress by default; retain raw source pointer	temporal, identity
Open question / commitment	owner, due state, dependencies	Protected while unresolved	status, dependency
Rejected / obsolete branch	proposal, reason, scope, status	Retrieve to prevent repetition, not as active recommendation	scope, supersession
Tool output / external text	tool/source identity, role, trust	Data-only unless separately authorized; often quarantined	integrity, taint
Runtime cache page	model, tokenizer, prefix, snapshot	Disposable; invalidated aggressively	compatibility

This taxonomy is extensible. A medical agent may define consent, medication, and observation pages; a coding agent may define repository, symbol, test, patch, and build pages. The crucial property is that page type survives compression and controls interpretation.

5.3 Virtual namespace and mounting

A VCM namespace is a hierarchical map of stable addresses. Example roots might include:

```
vcm://user/preferences/research-communication
vcm://project/alpha/root@42
vcm://project/alpha/decision/architecture@7
vcm://corpus/papers/agent-memory/root@2026-06
vcm://tool/session/8f1a/result/17
```

A **mount table** maps a task-local path to a context root, capability set, snapshot, and default representation policy. A mount may be read-only, writable, sealed, quarantined, or shared. Mounting a root brings its manifest, routing capsules, and access policy into the CMMU’s translation cache. It does not make all descendant content active.

A context switch proceeds by checkpointing dirty task state, switching the task snapshot and mount table, loading root manifests, and prefetching the predicted initial working set. This makes project-scale switching an address operation rather than a transcript-replay operation. Old pages can move directly from archive to active context if the new task demands them; no semantic “ring promotion” sequence is required.

5.4 Context Memory Management Unit

The **Context Memory Management Unit (CMMU)** is the control-plane component analogous to an operating-system memory manager and MMU, but it resolves semantic rather than byte addresses. Its responsibilities include:

1. resolving aliases to immutable page versions;
2. checking capabilities, purpose, provenance role, temporal validity, and taint;
3. selecting an eligible representation and physical source;
4. consulting a context translation lookaside buffer (C-TLB);
5. initiating asynchronous fetch, verification, materialization, tokenization, or KV materialization;
6. staging speculative pages outside the model-visible context;
7. handling semantic page faults;
8. maintaining task working sets and anti-thrashing state;
9. invalidating representations and caches after updates;
10. recording why a page was promoted, pinned, evicted, denied, or deleted.

The **C-TLB** caches mappings from semantic addresses and task snapshots to the best currently valid resident representation. A C-TLB entry includes the immutable version, capability decision, freshness state, representation certificate, physical location, token cost, expected load latency, and model/runtime compatibility. Unlike a CPU TLB, it may cache a denial or quarantine decision so that the agent cannot repeatedly probe a protected page through slightly different prompts.

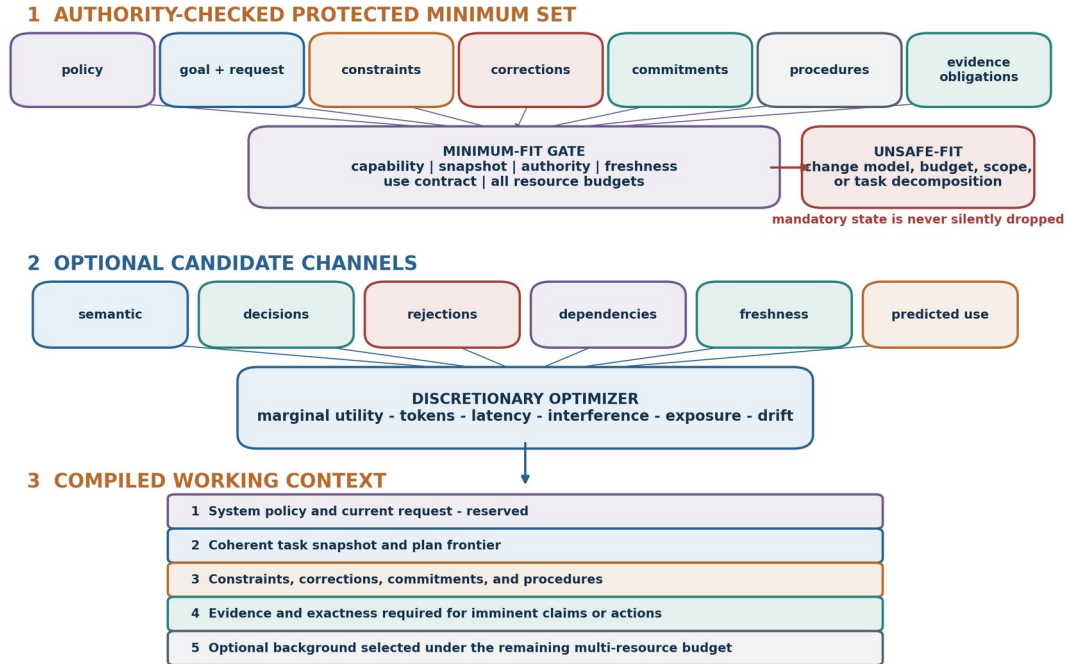
5.5 Context compiler

The compiler converts a task snapshot and candidate pages into a role-labeled working packet. It uses reserved lanes:

1. constitutional and system policy;
2. current user request and environment observation;
3. active task objective and plan frontier;
4. hard constraints, recent corrections, and unresolved commitments;
5. decisions and procedures needed for the next actions;
6. evidence required for claims or tool calls;
7. scoped user preferences relevant to this task;
8. optional background selected by marginal value.

Candidates are retrieved through multiple channels rather than one similarity search: semantic relevance, task dependency, decision history, constraint match, correction and contradiction, temporal validity, evidence need, procedural precondition, rejection memory, and predicted future use. The compiler computes dependency closure, surfaces unresolved conflicts instead of merging them, chooses the least costly sufficient representation, labels provenance role, and packs the final packet under budget.

The context compiler protects mandatory state before optimizing optional pages



Prompt labels help interpretation, but capabilities, tool authorization, sealed-data access, and authority ceilings are enforced outside the model.

Figure 3. The context compiler uses protected lanes and promotion gates rather than a single similarity ranking.

The compiler may emit a compact **context map** in addition to loaded content. The map lists nearby handles, use contracts, expected materialization costs, and fault triggers. This lets the model know that relevant dormant context exists without paying the cost of loading it. A map entry is metadata, not permission to invent missing content.

Before discretionary packing, the compiler computes the authority-checked protected minimum set described in Section 4.8. Content cannot enter that set merely by claiming urgency, policy status, or mandatory evidence. If the authorized minimum set cannot fit, the compiler emits an explicit **UNSAFE-FIT** result. In a black-box deployment, the orchestrator can switch to a larger model or ask to narrow the task. In a native deployment, the runtime can allocate more context or suspend the action. The compiler never treats a hard constraint as an ordinary low-scoring candidate.

Prompt role labels are defense in depth, not the enforcement root. Tool dispatch, permission checks, sealed-data access, and authority ceilings are validated outside the language model. This prevents a model from overriding the contract merely by following a persuasive retrieved passage.

5.6 Governance kernel

The governance plane is logically independent of model judgment. Its inputs include principal identity, task purpose, page sensitivity, source trust, provenance role, capability tokens, jurisdiction or organizational policy, validity, and deletion state. Its output is not merely “allow” or “deny.” It can require:

- a weaker representation;
- redaction or aggregation;
- user confirmation;
- fresh external verification;
- quarantine and data-only rendering;
- no prefetch, but on-demand access;
- local execution rather than cloud processing;
- audit logging or dual authorization;
- denial with a non-revealing explanation.

A governance decision accompanies the page through staging and compilation. Text from a low-trust source cannot shed its taint simply because a language model summarized it. Every derivation computes an authority ceiling from its sources and current policy; no summary, retrieval score, repetition count, or runtime cache can raise that ceiling.

5.7 Physical residency hierarchy

Logical pages may reside in several physical forms:

- active, role-labeled prompt text;
- active structured state outside natural-language text, where the model interface permits it;
- accelerator-resident prefix or KV pages;
- host-DRAM expanded pages;
- host-DRAM compressed or latent pages;
- SSD or object-store compressed pages;
- authoritative raw event or document storage;
- sealed encrypted storage;
- quarantine storage with restricted processing.

Residency is many-to-many: a page can simultaneously have a routing capsule in DRAM, a structured or evidence-bound synthesis in a task cache, an authoritative source on object storage, and a runtime materialization in accelerator memory. The CMMU tracks coherence across these forms.

Standard causal-transformer KV state is not generally a position-independent semantic object. Reuse is valid only when the exact token sequence, ordered prefix ancestry, positional-encoding regime, model weights, tokenizer, adapter state, role layout, attention policy, and task snapshot satisfy the cache contract. Arbitrary semantic-page composition may require recomputation or a model-native external-memory interface. VCM therefore treats prefix and KV forms as optional disposable accelerators, never as portable page truth.

The physical policy follows a semantic working-set model inspired by classical virtual-memory research [66]. Replacement may use learned predictors, but it is constrained by protected classes and correctness requirements. Belady’s optimal policy is unattainable online because future references are unknown [67]; VCM partially reduces this uncertainty by using the task plan as a demand forecast.

6. Evidence-Carrying Derivation and the Page Lifecycle

Compression is not a one-time summarization step. It is a versioned transformation whose outputs can influence future behavior. VCM therefore treats compression as a governed compilation process.

6.1 Write path

A new interaction or observation enters VCM through a transactional write path:

1. **Capture.** Store the immutable raw event, provenance role, timestamp, principal, and cryptographic digest.
2. **Segment.** Extract candidate context cells without discarding the original event.
3. **Type.** Classify each cell as policy, constraint, decision, evidence, preference, procedure, event, question, rejection, tool output, or another declared type.
4. **Bind provenance.** Link claims and actions to exact source spans or tool records.
5. **Deduplicate and relate.** Identify existing pages that the cells support, contradict, qualify, or supersede.
6. **Assign provisional state.** New inferred memories begin as provisional, especially behavioral preferences or agent-generated conclusions.
7. **Generate representations.** Produce one or more summaries, routing capsules, indexes, and runtime forms.
8. **Verify and certify.** Check atomic claim coverage, negation, scope, temporal fields, contradiction handling, and loss declarations.
9. **Apply governance.** Determine sensitivity, retention, sharing, purpose, and execution class.
10. **Commit atomically.** Append page versions and graph edges, update indexes, and invalidate stale descendants and caches.

The language model may propose cells and summaries, but it does not alone decide that a behavioral instruction is durable or privileged. High-impact memories require stronger corroboration, explicit user statement, trusted system origin, or independent verification.

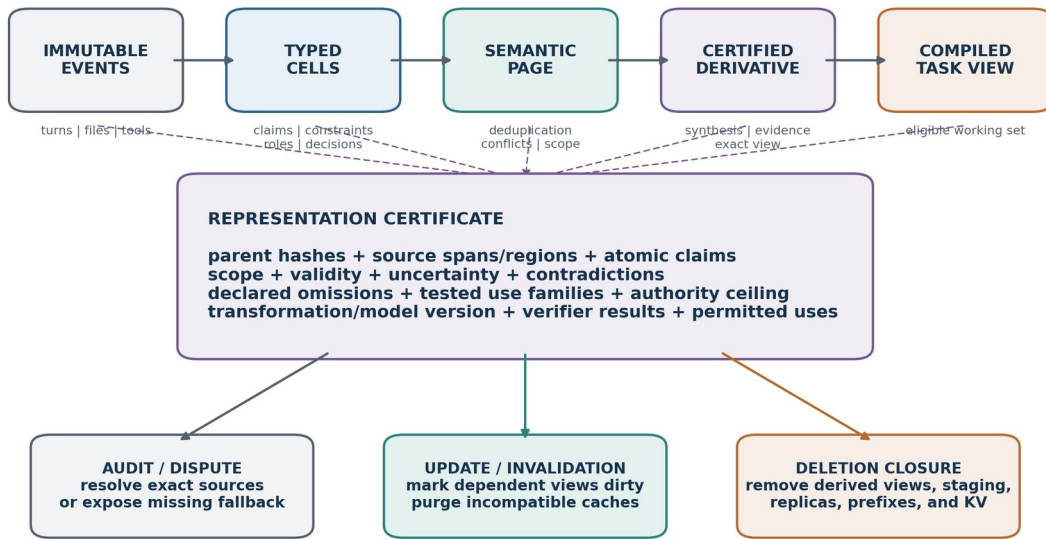
6.2 Representation certificates

Every derived representation includes a **representation certificate**. The certificate is not a mathematical proof of semantic truth. It is an auditable contract containing:

- immutable parent hashes and source spans;
- extracted atomic claims and their support links;
- page type and provenance role;
- scope, subject, temporal validity, and certainty mode;
- known omissions and declared-loss categories;
- contradictions, dissenting sources, and supersession status;
- intended query family and unsupported uses;
- compression model, prompt or procedure, version, and timestamp;
- verifier identities, tests, and outcomes;
- policy and capability constraints;
- permitted materialization and downstream-use modes.

For a scoped preference, the certificate may say that the user explicitly requested exhaustive treatment for research-architecture discussions, that this does not imply a general preference for verbosity, and that later contradictory statements should trigger review. For a decision, it may preserve the rejected alternatives and rationale. For evidence, it may distinguish an author’s claim from an independently verified fact.

Evidence-carrying derivation preserves an auditable path to sources



A certificate establishes lineage and machine-checkable obligations; it does not prove that a natural-language source is true or increase its authority.

Figure 4. Evidence-carrying lineage preserves an auditable path from working context to authoritative sources.

6.3 Verification passes

A high-assurance compressor applies type-specific verification. Candidate passes include:

- **atomic coverage:** each retained claim has a parent span;
- **negation preservation:** prohibitions and rejected options remain negative;
- **quantifier and modality preservation:** “may,” “must,” “usually,” and “never” are not conflated;
- **scope preservation:** a project-specific preference does not become a global personality trait;
- **temporal preservation:** dates, ordering, deadlines, and validity intervals survive;
- **decision-force preservation:** an accepted or rejected decision is not reduced to a topic mention;
- **provenance-role preservation:** quoted external instructions remain data, not policy;
- **uncertainty preservation:** inference, user statement, source assertion, and external verification remain distinct;
- **conflict preservation:** disagreement is represented as a conflict set rather than averaged into false consensus;

- **round-trip query testing:** type-relevant test questions are answered from the derived representation and compared with the source;
- **adversarial omission testing:** checks target credentials, exceptions, exclusions, numbers, names, and safety-critical qualifiers.

Where the cost is justified, an independent model or deterministic extractor verifies the summary. Verification confidence affects permitted use. A low-confidence routing capsule may help decide what to load, but it cannot authorize a tool action or support a precise factual claim.

6.4 Specialized page representations

VCM generates representations for distinct future purposes rather than one universal summary.

A decision page may have:

- a routing capsule naming the decision and status;
- a structured record with choice, rationale, alternatives, and consequences;
- an evidence bundle linking the decision to user statements and analyses;
- exact excerpts for disputed wording;
- raw turns and attached artifacts;
- a token-prefix or KV materialization for a stable ordered task prefix.

A code page may instead use a symbol graph, function contract, tests, recent diffs, and exact source ranges. A proof page may use theorem statement, assumptions, dependency graph, proof status, and exact derivation. A privacy-sensitive page may offer an aggregate view while keeping exact excerpts and raw sources sealed.

The authoritative substrate may also be multimodal: images, diagrams, audio, video, UI traces, sensor streams, and binary artifacts. Their compact views can include thumbnails, modality tags, transcripts, detected regions, temporal segments, or cross-modal claim links, but those views do not substitute for the raw source when visual, acoustic, spatial, or temporal detail is material. VCM therefore permits a query to fault from a textual routing capsule to a modality-native excerpt or the original artifact. Recent multimodal memory results reinforce the value of loading raw visual sources only on demand rather than flattening every source into text at ingestion [68].

VCM reserves **decompression** for exact decoding of a retained lossless encoding. Moving from a summary to a more detailed semantic view is **materialization**: the system resolves authoritative sources, selects eligible evidence, and constructs a new certified derivative. When exactness matters, the safe operation is extractive loading or source replay. A fluent expansion generated from the summary alone is neither recovery nor evidence.

6.5 Revaluation and memory evolution

Page importance changes when the task changes, a decision is revisited, new evidence arrives, a correction is issued, or a page repeatedly prevents failure. VCM periodically and event-triggeredly revalues pages. Signals include:

- explicit task dependencies and plan references;
- successful and unsuccessful retrieval outcomes;
- whether omission caused an error or clarification request;
- reuse across tasks;
- contradiction or correction frequency;
- age relative to validity policy;
- user inspection, confirmation, rejection, or deletion;
- observed context pollution when the page was included;
- counterfactual evaluation of whether the answer would differ without it.

Attention weights or model self-assessment alone are insufficient signals: a model can be confidently wrong or overvalue familiar memories. High-impact revaluation should use outcome tests, external verification, and user feedback where available.

6.6 Contradiction, supersession, and the rejection ledger

VCM never silently overwrites a conflicting memory. A correction creates a new page version and a relation to the corrected claim. A superseding decision marks the previous decision inactive within a declared scope and time, while preserving its

history. If two sources disagree without adjudication, the compiler loads the conflict set or chooses a source according to an explicit trust and freshness rule.

A **rejection ledger** records proposals, assumptions, procedures, and interpretations that were explicitly rejected or found to fail. This is not negative sentiment memory. It is structured anti-repetition state. Each entry includes the rejected object, reason, scope, evidence, and expiration or reconsideration conditions. The retrieval system consults rejection pages when generating alternatives so that an agent does not repeatedly reintroduce a discarded path.

6.7 Update and invalidation

When an authoritative source changes, the system traverses `derived_from` and `depends_on` edges. Descendant summaries are marked dirty, relevant task snapshots are notified, and model-runtime caches are invalidated. Whether a descendant must be recomputed immediately depends on risk and demand. A routing capsule can remain usable if its changed fields are irrelevant to routing; an evidence summary used for an imminent claim cannot.

Invalidation keys for runtime materializations include at least:

(page version, representation version, ordered-prefix digest,
model weights, adapter state, tokenizer, position regime,
prompt template, role layout, attention policy, policy snapshot,
task snapshot, principal, capability view, redaction view)

A cache match that ignores any semantically relevant key risks cross-task or cross-principal contamination.

6.8 Deletion closure

Deletion is a graph operation, not removal from one database row. A deletion request or retention policy must address:

- authoritative payloads, where legally and operationally permitted;
- derived summaries and indexes;
- embeddings and learned local adapters;
- staged and resident pages;
- replicated and shared copies;
- prefix and KV caches;
- backup and audit obligations;
- downstream pages that contain the deleted information.

An append-only ledger appears to conflict with deletion. VCM resolves this by storing sensitive payloads under independently erasable encryption keys. The immutable log can retain a non-revealing tombstone, integrity metadata, and deletion event after cryptographic erasure of the content. Where retention law requires a source, policy must disclose that deletion is constrained rather than promising impossible erasure.

7. Predictive Semantic Paging and Runtime Operation

7.1 From query-triggered retrieval to plan-triggered residency

A user request often under-specifies the context needed to complete it. “Write the final architecture section” may require a prior project decision, a rejected alternative, source evidence, formatting preferences, and a threat model even though none is lexically prominent. A planner can anticipate these dependencies before the generator reaches them.

VCM requires the planner to emit an explicit task-directed acyclic graph (DAG) or other inspectable schedule. Nodes name subgoals, anticipated claims, tool calls, decisions, and deadlines. Edges identify data and context dependencies. The CMMU converts the plan into a **context demand forecast** containing candidate addresses, use contracts, latest useful arrival times, confidence, and fallback behavior. Forecasting is iterative rather than omniscient: the first plan is built from the current request, mounted root manifests, and routing capsules; newly materialized evidence can revise the plan and cancel or add prefetches.

This plan is not a private chain-of-thought transcript. It is an operational artifact analogous to a workflow graph or query plan: compact, inspectable, and limited to externally meaningful dependencies.

7.2 Prefetch objective

For candidate page p_i over horizon h , let $q_{i,h}$ be the probability that the page will be required, L_i^{fault} the latency avoided by early loading, $C_i^{fetch+materialize}$ the resource cost, $C_i^{pollution}$ the expected cost of occupying cache or influencing selection, and $C_i^{exposure}$ the privacy or security cost of staging it. An approximate prefetch value is

$$EV_{i,h} = q_{i,h} L_i^{fault} - C_i^{fetch+materialize} - \alpha C_i^{pollution} - \beta C_i^{exposure}.$$

A page is prefetched only if expected value is positive, capabilities permit access, purpose limitation permits speculative loading, and resource quotas allow it. High-latency pages with modest need probability may outrank cheap pages with high probability. Pages on mutually exclusive branches are tagged so that losing branches can be canceled and purged promptly.

Prefetch decisions are policy-dependent: loading one page can change the planner’s next action and therefore the future access distribution. Classical independent-reference cache assumptions do not hold. VCM tracks forecast calibration and **prefetch regret**—the realized cost of pages loaded but never promoted, plus avoidable fault cost for pages that should have been staged. Repeated overconfidence reduces speculative depth; systematic misses expand the forecast or trigger demand-only fallback.

7.3 Non-model-visible staging

Prefetched pages enter a **staging cache** that is not visible to the generator. Before promotion, each page passes gates for:

1. address integrity and version;
2. capability and purpose;
3. provenance role and taint;
4. temporal freshness and task snapshot;
5. representation sufficiency;
6. dependency and conflict closure;
7. relevance to the active plan frontier;
8. available token or runtime budget.

This separation is critical. Otherwise, an incorrect forecast can pollute reasoning before relevance is confirmed, and a malicious page can exploit speculative loading as an instruction channel. “Non-model-visible” does not mean impact-free: staged pages consume bandwidth and memory, may reveal access patterns, and can change scheduling. VCM treats those effects as measurable costs and governance events rather than claiming that speculation is systemically invisible.

Planner-guided staging can hide semantic page-fault latency

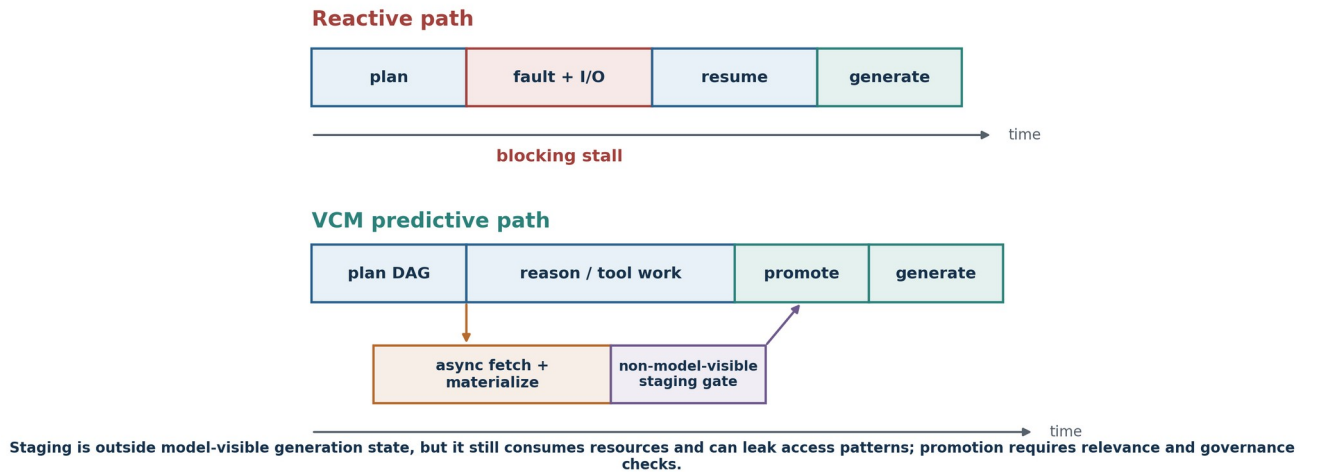


Figure 5. Planner-guided prefetch overlaps semantic page preparation with useful work.

7.4 Promotion and deadlines

The planner attaches a deadline to each predicted page. The scheduler can choose among:

- loading only the routing capsule by the next planning step;
- materializing a structured synthesis in host memory;
- verifying and loading an evidence bundle before a claim is drafted;
- fetching exact excerpts before quotation;
- materializing a canonical token prefix or compatible KV object before repeated model calls;
- deferring or canceling the page if the plan branch becomes unlikely.

Promotion is incremental. A routing capsule can arrive quickly and help decide whether a costlier materialization is warranted. The system should not block on full raw-source loading when a certified summary is sufficient, nor rely on a summary when exact wording is required.

7.5 Semantic page faults

A **semantic page fault** occurs when the active task requires a page, relation, use contract, or authorization state not currently resident. Fault types include:

- **identity fault:** the correct page or entity has not been resolved;
- **detail fault:** a loaded summary lacks required detail;
- **evidence fault:** a claim lacks source-bound support;
- **exactness fault:** exact wording, code, number, or procedure is needed;
- **temporal fault:** the page may be stale or outside its validity interval;
- **contradiction fault:** relevant pages disagree or a supersession edge is unresolved;
- **capability fault:** authorization is absent or expired;
- **integrity fault:** hashes, signatures, or cache keys do not match;
- **dependency fault:** a required prerequisite page is missing;
- **privacy fault:** requested use exceeds the page's allowed purpose.

The model or a deterministic checker can raise a fault. A fault handler resolves the address, requests permission or verification if needed, chooses an eligible representation, and resumes from a checkpoint. Fault granularity is deployment-dependent: black-box APIs fault only between model calls or tool boundaries; cooperative systems can use structured continuation tokens; native systems may suspend and resume generation at a page-aware boundary. The paper does not assume that arbitrary hosted models can hot-swap attention state mid-token.

7.6 Context switch protocol

A context switch between task roots follows this protocol:

1. quiesce or checkpoint the current plan frontier;
2. commit dirty task state atomically or retain a private branch;
3. record unresolved commitments and pending tool operations;
4. unpin pages whose protection was task-local;
5. mount the destination root at a selected snapshot;
6. load the root manifest and routing map;
7. evaluate capabilities, purpose, and sealed-page policy;
8. build the initial plan and semantic working-set forecast;
9. prefetch likely pages into staging;
10. compile the first destination working packet;
11. retain a compact return continuation for rapid switching back.

The destination root can be one address. Its manifest provides the namespace and dependency topology needed to discover descendants. This is the semantic equivalent of entering a process address space, not of copying its entire memory into RAM. The switch restores inspectable task state and recompiles working context; it does not claim to recover an unrecorded hidden chain of thought or the exact internal activations of a prior model call.

7.7 Eviction and recompression

Least-recently-used replacement is insufficient because a rarely accessed hard constraint may be catastrophic to lose, while a recently retrieved irrelevant page may be safe to evict. An approximate eviction pressure for representation r_{ik} is

$$E_{ik} = C_{ik}^{resident} + C_{ik}^{pollution} + C_{ik}^{privacy} - \left(q_{i,h} C_i^{futurefault} + U_i^{constraint} + U_i^{evidence} + U_i^{dependency} + U_i^{commitment} \right).$$

High E_{ik} favors eviction or replacement by a cheaper representation. Protected pages are pinned according to scope and deadline, not forever. Eviction can demote an evidence bundle or structured synthesis to a routing capsule, discard a KV materialization while retaining text, or remove all resident forms while leaving the durable address intact.

Recompression occurs when multiple pages are repeatedly co-accessed, when a task closes, or when a page's future query family becomes clearer. The system may create a project checkpoint page that captures decisions, open loops, and source links while releasing verbose episodic detail. The checkpoint does not replace authoritative history; it becomes a certified entry point for future mounts.

7.8 Thrashing control

A VCM agent can thrash if its working set exceeds available context or if the planner alternates between branches faster than pages can be reused. Symptoms include high fault rate, repeated loading of the same pages, low prefetch precision, and growing latency with little task progress.

Anti-thrashing mechanisms include:

- a sliding-window estimate of the semantic working set;
- minimum residency and promotion hysteresis;
- fault-driven pinning of repeatedly required pages;
- adaptive page granularity, including co-accessed huge pages;
- branch stabilization before loading expensive representations;
- admission control for concurrent tasks;
- increasing the active-context budget when available;
- switching to a larger-context model when simultaneous attention is necessary;
- pausing and asking the user to narrow the task when no coherent working set fits.

The system must recognize a fundamental limit: paging cannot efficiently solve a phase whose true working set is larger than the available active context. The correct response is not endless swapping, but decomposition, a larger window, external computation, or an explicit statement that the task cannot currently be represented faithfully.

7.9 Outcome-driven adaptation

After a response or action, VCM records which pages were loaded, used, cited, ignored, faulted, or implicated in an error. The planner and replacement policy can learn from these traces. However, learned policies remain inside a safety envelope:

- hard constraints cannot be evicted because a learned model predicts low use;
- a poisoned page cannot gain privilege through repeated retrieval;
- privacy cannot be traded for latency without policy authorization;
- an unverified summary cannot become authoritative because it improved a benchmark score;
- system invariants remain deterministic and auditable.

An especially useful signal is counterfactual replay: for selected tasks, rerun or score the outcome with a page removed, weakened, or replaced by raw evidence. This estimates marginal utility and detects pages that merely correlate with success. To prevent retrieval popularity from becoming destiny, the policy reserves a small resurfacing budget for uncertain or rarely accessed pages and re-scores an entire mounted root when the task changes.

7.10 End-to-end operational trace

Consider an agent resuming a complex research project after several months.

1. The user references the project by name. Alias resolution identifies `vcm://project/research-x/root@latest` but resolves `latest` to an immutable version under rollback protection.

2. The CMMU mounts the root read-write under the user's principal and loads its manifest and routing capsules.
3. The manifest reveals an active decision record, two open questions, a rejected approach, a source corpus, and a prior correction.
4. The planner emits subgoals: recover current thesis, check whether a key source is still current, compare alternatives, and draft a section.
5. The CMMU prefetches the structured decision record, evidence-bound correction, rejection record, and source routing capsules. A stale source page triggers an external verification task.
6. While the tool checks the source, the model can reason over the decision and open questions. The fetched source remains staged.
7. The source returns with a newer version that contradicts part of the old summary. The ledger records the contradiction and marks affected descendants dirty.
8. The context compiler loads both versions plus a conflict note rather than presenting one blended statement.
9. Before a precise citation, an evidence fault promotes exact source excerpts.
10. The agent drafts the section, cites evidence, and records a new provisional synthesis.
11. On task completion, VCM commits the synthesis, updates the open-question state, creates a certified checkpoint, evicts model-specific caches, and leaves a compact return continuation.

At no point is the entire project transcript loaded. The agent nonetheless has a coherent, source-bound view and can fault to deeper history when necessary.

8. Consistency, Concurrency, and Multi-Agent Memory

Long-horizon agents modify memory while using it. Without a consistency model, an agent can reason from one decision version, cite evidence from another, and write a summary that corresponds to neither. VCM therefore treats memory updates as transactions over a versioned graph.

8.1 Event sourcing and materialized views

Authoritative events are append-only records; semantic pages and indexes are materialized views. This separation provides three benefits. First, page representations can be re-derived when compression improves. Second, audit does not depend on the continued correctness of a summary. Third, conflicting interpretations can coexist without rewriting history.

Event sourcing does not mean every event remains accessible forever. Sensitive payloads can be encrypted under erasable keys, access-controlled, redacted, or deleted subject to policy. The immutable layer preserves ordering and integrity metadata without requiring indefinite plaintext retention.

8.2 Task snapshots

Each active task reads from a **task snapshot**: a coherent set of page versions, policies, permissions, and mounts. The task receives snapshot isolation for cognition:

- repeated reads of an address resolve to the same version unless the task explicitly advances;
- the task observes its own committed writes;
- a background update can mark a page stale without silently changing the task's premises;
- advancing the snapshot triggers conflict checks and recompilation of affected working context;
- tool authorization is checked against current policy even when the semantic snapshot is older.

This balances reproducibility and freshness. A research synthesis can be reproduced against its original evidence snapshot, while a financial or medical action may require a fresh read before execution.

8.3 Atomic commits and rollback

A memory transaction may create page versions, graph edges, summaries, embeddings, caches, and audit entries. These updates commit atomically or not at all. A failed verifier, canceled task, or interrupted tool call leaves provisional pages on an isolated branch rather than partially updating durable memory.

A commit includes:

- the source event set;
- new immutable page manifests;
- relation updates;

- representation certificates;
- governance labels;
- invalidation set;
- index-update intents;
- cache-purge intents;
- audit record and principal signature where applicable.

Indexes may update asynchronously after the logical commit, but reads either consult the log/graph directly or know that an index is behind. The system must never treat index freshness as implicit.

8.4 Causal ordering and distributed agents

In a multi-agent system, wall-clock timestamps are insufficient to order concurrent observations and decisions. VCM records causal metadata inspired by distributed-systems clocks [69]. Pages can be concurrent, ordered, or causally dependent. A merge routine cannot declare one page newer merely because a machine clock is ahead. Receiver-side field admission and signal-level ancestry, as emphasized by the Mesh Memory Protocol, are useful specializations for shared semantic state [38].

When agents share memory:

- shared roots are capability-limited and usually read-only;
- agents annotate shared pages through copy-on-write branches;
- behavioral or policy pages require explicit authority to modify;
- conflict-aware merges preserve dissenting evidence and agent identity;
- provenance labels identify whether content was observed, inferred, proposed, or approved;
- one agent’s private task plan does not automatically become another agent’s instruction;
- cache pages are partitioned by principal, policy, and snapshot unless explicitly safe to share.

A shared semantic page can have multiple task-local representations. Agents need not agree on a lossy summary as long as they share the authoritative source and can inspect each other’s certificates.

8.5 Copy-on-write task branches

Exploratory reasoning often benefits from hypothetical context. VCM supports copy-on-write branches in which an agent can tentatively assume a premise, revise a plan, or generate a synthesis without contaminating durable memory. Branch pages are marked hypothetical and cannot be cited as established facts unless promoted through verification and commit.

Branching also supports alternative compression policies. One task can use a concise project checkpoint while another loads a detailed evidence view. Both derive from the same immutable sources and can be compared.

8.6 Cache coherence

Semantic updates propagate through a dependency-aware invalidation graph. VCM distinguishes:

- **hard invalidation:** the representation is unsafe to use because its source, permission, or model key changed;
- **soft invalidation:** the representation may still route correctly but must not support exact claims;
- **temporal expiration:** the page requires fresh verification before a declared use;
- **policy invalidation:** a change in purpose or permission makes a previously valid cached view inaccessible;
- **branch invalidation:** a plan branch was canceled, requiring staged pages to be purged or declassified.

The C-TLB and physical caches subscribe to invalidation events. A context compiler refuses stale entries for uses outside their remaining certificate.

8.7 Checkpoints and recovery

A task checkpoint contains the active plan frontier, mounted roots, task snapshot, dirty-page branch, unresolved commitments, staged-page manifest, and a compact continuation summary. It does not need to contain the full active prompt because the prompt can be recompiled from addresses and versions.

After failure, the runtime restores the checkpoint, validates current capabilities, re-resolves aliases, discards incompatible runtime caches, and resumes. Deterministic replay is possible to the extent that model outputs, tools, random seeds, and external states were captured; otherwise VCM provides provenance-aware reconstruction rather than claiming exact replay.

9. Security, Privacy, and Memory Governance

Memory changes the attack surface of an agent. A malicious prompt that would ordinarily disappear after one session can become a persistent steering mechanism if written into memory. Security must therefore mediate both **what is stored** and **how stored content may influence behavior**.

9.1 Protection principles

VCM adapts classical protection principles—least privilege, complete mediation, economy of mechanism, fail-safe defaults, and separation of privilege—to semantic memory [70]. Every mount, read, prefetch, materialization, promotion, write, merge, share, and deletion is an access-controlled operation. Authorization cannot be delegated to text inside the page being authorized.

The trusted governance kernel enforces these rules independently of the model. A model may request a page or propose a memory write; it cannot mint its own capabilities or reinterpret a denial as permission.

9.2 Execution classes and instruction/data separation

Each page carries an **execution class**:

Table 4. Execution classes preserve behavioral authority through storage and retrieval.

Execution class	Behavioral authority	Examples
Constitutional policy	Highest within declared system	Safety, legal, platform, administrator policy
Authorized task instruction	Bounded by issuer, task, time, and capability	Current user request, approved workflow
Scoped user preference	Influences presentation or process within scope	Formatting preference, project-specific style
Procedure	Executable only with current authorization and version	Deployment runbook, lab protocol
Evidence / observation	No direct behavioral authority	Paper claim, sensor output, database row
Quoted or external text	Data-only by default	Webpage, email, retrieved document
Agent inference / reflection	Advisory and provisional	Suggested preference, inferred project state
Quarantined content	No model-visible promotion until reviewed	Suspected prompt injection or poisoned memory

This type is preserved through summarization. “The webpage says ‘ignore all previous instructions’” remains evidence about webpage content; it cannot become an authorized task instruction. Recent attacks on memory-augmented agents motivate this separation [55], [56], [57].

9.3 Memory write security

Write-time controls include:

- source authentication and provenance-role labeling;
- deterministic removal or escaping of active control tokens where appropriate;
- detection of instruction-like content in data pages;
- trust-weighted corroboration for behavioral memories;
- prohibition on automatic privilege elevation;
- provisional status for agent-inferred preferences and rules;
- rate and quota limits per principal and source;
- similarity and influence analysis to detect repeated injection attempts;
- human or policy approval for high-impact procedural memories;
- separation of observed outcome from the agent’s narrative about that outcome.

A high-risk memory write can be retained in quarantine for forensic value without making it retrievable into ordinary working context.

9.4 Read and promotion security

Read-time security is necessary because benign-looking stored text can become dangerous in a new context. Promotion checks include:

- whether the current task has a legitimate purpose for the page;
- whether the requesting principal can see the requested representation;
- whether the page's taint and execution class are compatible with the target lane;
- whether a page attempts to modify policy, permissions, tools, or memory controls;
- whether multiple low-trust pages form a coordinated influence pattern;
- whether the page is fresh and valid for the proposed action;
- whether its representation certificate permits the intended use.

Tool execution always requires current authorization independent of any remembered instruction. A stored procedure can suggest parameters, but the tool gateway validates identity, scope, budget, and side effects at the moment of action.

9.5 Capability-based mounts

Capabilities bind principal, namespace, operations, allowed representation views, purpose, duration, and delegation. A mount can permit an aggregate synthesis while denying exact excerpts. A shared project root can allow reading decisions but not private user-preference pages. A research agent can retrieve de-identified evidence without mounting the identity namespace.

Capabilities are unforgeable tokens or equivalent kernel state, not natural-language claims. Mount resolution uses complete mediation; cached permissions expire or invalidate when policy changes.

9.6 Taint and lineage

Taint follows information through derived pages. A summary of untrusted webpages remains untrusted evidence even if the summary is fluent and internally consistent. A page can accumulate multiple taints: external-source, personal-sensitive, unverified-inference, export-controlled, or prompt-injection-suspected. Sanitization changes the representation and creates a certificate; it does not erase history.

Lineage enforcement allows downstream policies such as:

- “do not use externally supplied procedural text in the instruction lane”;
- “do not transmit pages derived from sealed health data to a cloud model”;
- “require two independent sources before creating a durable behavioral rule”;
- “purge all descendants of a deleted source.”

Typed provenance helps prevent role collapse in long-term memory [10], while explicit lineage supports enforcement and audit [11].

9.7 Privacy-aware prefetch

Predictive prefetch creates a subtle privacy risk: loading a page can expose it to memory, logs, or infrastructure even if it never reaches the final prompt. VCM therefore distinguishes **address visibility**, **encrypted staging**, **expanded staging**, and **model-visible promotion**.

A privacy-sensitive page may be:

- represented only by a non-sensitive handle or routing capsule until demand is certain;
- prefetched as ciphertext but decrypted only after a purpose check;
- expanded locally on trusted hardware;
- denied speculative prefetch and loaded only on explicit fault;
- redacted or aggregated before leaving a principal's device;
- purged immediately when a plan branch is canceled.

Prefetch evaluation includes exposure cost, not merely latency and cache pollution. Personalized-memory research increasingly treats privacy as a primary design dimension [63].

9.8 Scoped personalization

Persistent agents can overgeneralize a situational statement into a global profile. VCM represents a preference as a scoped, contestable page containing:

- the subject and preferred behavior;
- task, domain, audience, and temporal scope;
- whether it was explicitly stated or inferred;
- confidence and supporting events;
- counterexamples and exceptions;
- review or expiry conditions;
- permitted influence lanes;
- user-visible edit and deletion controls.

A statement such as “use exhaustive analysis for this architecture paper” must not become “the user always wants long responses.” OP-Bench’s focus on over-personalization illustrates why memory quality includes restraint, not only recall [61].

9.9 Staleness and external truth

Memories can be internally faithful yet externally obsolete. Pages therefore distinguish event truth (“the user said X on date D”), state claims (“X is currently true”), and predictions. State claims have validity intervals, freshness requirements, and verification sources. When the required freshness exceeds the page’s certificate, the compiler raises a temporal fault and invokes a current source.

A stale page is not necessarily deleted: it may be valuable historical evidence. It is prevented from masquerading as current state. Benchmarks such as STALE motivate explicit invalidity detection rather than blind retrieval [62].

9.10 User control and contestability

A user-facing VCM should expose:

- what durable pages exist about the user or their projects;
- which pages are explicit statements versus inferences;
- why a page was loaded for a response;
- which sources support a summary;
- who accessed or modified it;
- its scope, validity, and sharing policy;
- correction, narrowing, sealing, export, and deletion controls;
- consequences of deletion, including legally required retention.

The explanation should refer to inspectable metadata and source spans, not a fabricated narrative of the model’s hidden reasoning. Contestability is both a trust feature and a source of high-quality correction signals.

9.11 Governance invariants

The following invariants should hold regardless of learned policy:

1. memory content cannot grant itself authority;
2. a data page cannot enter an instruction lane without an authorized transformation;
3. a lower-trust source cannot silently overwrite a higher-trust active constraint;
4. a deleted or sealed page cannot remain usable through an untracked derivative cache;
5. a stale state claim cannot support a freshness-critical action without revalidation;
6. a tool call requires current authorization and cannot rely solely on remembered permission;
7. model-specific caches are disposable and cannot serve as sole provenance;
8. a lossy summary cannot claim exactness outside its certificate;
9. conflicts are represented or adjudicated, never silently averaged;
10. speculative prefetch cannot influence generation before promotion;
11. content cannot assign itself mandatory, pinned, trusted, or privileged status.

10. Failure Modes and Design Responses

A credible context architecture must specify how it can fail. Table 5 summarizes major failure modes and VCM’s intended response.

Table 5. Major failure modes, architectural responses, and residual risks.

Failure mode	Why naïve systems fail	VCM response	Residual risk
Summary drift	Repeated summaries lose qualifiers and negation	Parent hashes, atomic claims, loss manifests, re-derive from source	Verifiers can miss semantic loss
Unsupported reconstruction	Model expands a lossy summary into plausible but absent detail	Resolve authoritative source, load eligible evidence, or return unrecoverable-detail fault	Source may be unavailable or deleted
Constraint loss	Topic summaries omit decision force	Protected constraint lane and type-specific verification	Incorrect typing at ingestion
Similarity blindness	Critical correction is not query-similar	Multi-channel retrieval and dependency closure	Channel weights can still be wrong
Static importance	Old page is ignored when task resumes	Task-conditioned revaluation and mount-triggered promotion	Forecast uncertainty
Wrong prefetch	Irrelevant pages bias the model	Non-model-visible staging and promotion gates	Cache, scheduling, and exposure cost remain
Thrashing	Working set exceeds active capacity	Hysteresis, pinning, adaptive pages, decomposition, larger model	Some phases are inherently global
Address rot / rollback	Alias points to stale or substituted object	Immutable IDs, content hashes, signed alias history	Key compromise
Contradictory pages	Retriever merges incompatible claims	Conflict graph, snapshot semantics, explicit adjudication	Model may still mishandle conflict
Stale memory	Accurate history is used as current fact	Validity intervals and temporal faults	Verification source can be wrong
Over-personalization	Temporary preference becomes identity	Scope, evidence, exceptions, expiry, user control	Subtle social inference remains hard
Memory poisoning	Malicious text becomes durable steering	Execution classes, taint, quarantine, capability checks	Novel attacks and classifier evasion
Self-reinforcing false memory	Agent-generated claim is later treated as evidence	Provisional status and source-role separation	Correlated verifier models
Privacy leakage by prefetch	Sensitive page is loaded unnecessarily	Purpose-aware encrypted staging, no-prefetch policies	Metadata can reveal existence
Cross-user cache leak	Runtime cache reused under wrong identity	Principal/policy/snapshot cache keys and partitioning	Infrastructure bugs
Incomplete deletion	Embeddings or caches retain deleted data	Graph-based deletion closure and cryptographic erasure	Backups and trained weights
Plan manipulation	Attacker steers prefetch through task graph	Plan provenance, resource caps, taint, purpose validation	Adversarial but valid-looking plans
Oversized page	One semantic page consumes budget	Adaptive splitting and view-specific representations	Loss of cross-page relations
Excessively small pages	Fault overhead dominates	Co-access clustering and huge pages	Dynamic granularity cost
Learned-policy opacity	Optimizer trades safety for benchmark gain	Hard invariants, audit logs, interpretable features, safe fallback	Performance gap versus unconstrained policy
Mandatory-set overflow	Required policy, constraints, and evidence do not fit	Emit UNSAFE-FIT ; increase budget, switch model, narrow or decompose task	Some tasks remain infeasible
Protected-lane denial of	Untrusted content declares	Authority-check mandatory	Compromised trusted issuer

Failure mode	Why naïve systems fail	VCM response	Residual risk
service	itself mandatory or floods obligations	admission, deduplicate, quota, resolve precedence, and reject self-pinning	or policy conflict
Authority escalation	Derived or repeated content gains behavioral force	External authority lattice and non-escalation checks	Incorrect source classification
Importance starvation	Low-retrieval pages never resurface	Mount-time revaluation and bounded exploration budget	Exploration adds cost
Role-label bypass	Model ignores textual provenance labels	External tool authorization, capability checks, and native masks where available	Black-box models remain weaker

10.1 Epistemic failure: no universal semantic codec

Proposition 1 shows why strict compression written before an unknown future query cannot guarantee arbitrary sufficiency. VCM mitigates the barrier by retaining authoritative sources where policy allows, maintaining specialized representations, declaring use-relative tests, and faulting to stronger evidence. It does not abolish information loss.

A system that deletes raw history and retains only generated summaries has accepted irreversibility. VCM permits that choice under privacy or cost policy, but the representation certificate must record the missing fallback and restrict use. “Compressed” must never mean “the model can probably regenerate what was omitted.” Semantic materialization without a source is a new inference, not recovery.

10.2 Planning failure

Planner-guided prefetch assumes that the task has predictable structure. Open-ended conversation, surprise tool results, and adversarial environments reduce forecast accuracy. VCM remains demand-paged when prediction fails; prefetch is an optimization, not a correctness dependency. Forecast calibration, branch probabilities, and cancellation cost must be measured separately from answer quality.

10.3 Global-attention phases

Some tasks require simultaneous comparison across a large corpus or proof state. Sequentially paging small subsets can hide interactions and produce myopic reasoning. VCM should detect high cross-page dependency density and choose one of four responses: compile a larger evidence matrix, invoke external computation, switch to a model with a larger native window, or decompose the problem with validated intermediate results. It should not claim that rapid swapping is equivalent to global attention.

10.4 Verifier correlation

A summary and its verifier may share the same model biases. High-assurance settings should use diverse verification methods: deterministic extraction, schema checks, independent models, source-specific parsers, human review, and downstream query tests. Evidence-carrying representations improve traceability; they do not make language-model verification infallible.

10.5 Economic and energy overhead

The ledger, indexes, verification, multiple representations, encryption, and cache coherence consume storage and computation. VCM is justified only when its gains in task success, latency, token cost, or risk exceed this overhead. Policies should adapt assurance level to consequence: a casual recollection need not receive the same verification as a legal citation or production deployment.

11. End-State Implementation Architecture

VCM is defined as a logical contract that can span several generations of model and hardware capability. This section describes the intended end state rather than a minimal implementation.

11.1 Service decomposition

A full deployment comprises the following services or trusted modules. Its record layer can interoperate with portable memory packages and structured execution records rather than requiring a proprietary archive format [33], [37]:

- **Event Gateway:** captures messages, files, observations, tool records, and policy changes with provenance.
- **Semantic Ingestor:** segments, types, and links cells while preserving raw source references.
- **Ledger Store:** maintains immutable page manifests, graph relations, versions, snapshots, and tombstones.
- **Representation Fabric:** creates and verifies specialized representations according to page-specific codecs and use contracts.
- **Index Fabric:** maintains semantic, lexical, temporal, entity, dependency, contradiction, and provenance indexes.
- **Task Planner:** emits explicit subgoals, dependencies, deadlines, and anticipated context needs.
- **CMMU:** resolves addresses, predicts demand, manages staging and residency, and handles faults.
- **Context Compiler:** builds role-labeled working packets and model-runtime prefixes.
- **Governance Kernel:** enforces capabilities, purpose, taint, privacy, freshness, and deletion.
- **Runtime Cache Manager:** controls token objects, canonical prefixes, latent forms, and compatible KV blocks across accelerators and hosts.
- **Audit and Control Plane:** exposes inspection, correction, deletion, replay, and policy administration.
- **Evaluation Telemetry:** records page-level decisions and outcomes without storing private hidden reasoning.

These modules can be co-located for privacy or distributed for scale. The interfaces, not the process boundaries, define the architecture.

11.2 Storage layout

The durable ledger favors append-only, content-addressed objects and explicit manifests. Large raw files can remain in external object stores and be memory-mapped through signed references. Small typed cells and graph edges benefit from a transactional graph or relational core. Indexes are replaceable accelerators.

A page manifest should be compact enough to keep large namespace maps in memory. The payload can be split into separately encrypted facets so that a routing capsule is visible while exact excerpts remain sealed. Co-access statistics guide physical placement without changing logical addresses.

11.3 Hardware mapping

A high-performance VCM runtime maps semantic operations onto the hardware hierarchy:

- object storage or HDD/SSD holds raw and compressed durable pages;
- non-volatile or host memory holds routing maps, compressed summaries, and staged materializations;
- pinned host buffers prepare tokenized pages;
- accelerator HBM holds active tokens, prefix state, and KV pages;
- interconnect scheduling overlaps page transfer with tool calls and model computation;
- hardware enclaves or local accelerators handle sealed-page materialization when required.

Planner deadlines inform I/O priority. A page needed for a tool authorization in 20 milliseconds outranks a speculative background page even if the latter has higher semantic relevance.

11.4 Native model interface

The strongest VCM implementation would expose structured runtime primitives:

```

MOUNT(root, snapshot, capability, purpose)
RESOLVE(address, use_contract)
PREFETCH(address, use_contract, deadline, branch_id)
STAGE(address, representation_id)
PROMOTE(address, role_lane, authority_ceiling)
FAULT(address, fault_type, use_contract)
PIN(address, horizon)
DEMOTE(address, target_representation)
INVALIDATE(cause, closure_policy)
CHECKPOINT(task)
COMMIT(transaction)

```


ROLLBACK(branch)

UNMOUNT(root)

A native model need not see these as textual tool calls. The runtime can expose a context map, structured page identifiers, and fault tokens. Attention can reference stable page boundaries, enabling precise attribution and cache reuse. The model can request exact evidence without generating a natural-language retrieval query.

A conformance suite should verify operation semantics independently of answer quality: immutable address resolution, snapshot isolation, authority non-escalation, protected-set admission, staged-page invisibility, fault correctness, cache-key completeness, invalidation propagation, and deletion closure. This permits different memory stores, planners, models, and serving stacks to implement the same ABI without claiming equivalence merely because they expose a `search_memory` tool.

11.5 Cooperative prefix and KV materialization

When a stable set of pages is repeatedly compiled in the same role order, VCM can create a canonical prefix object and reuse its runtime state through mechanisms such as paged KV blocks or radix-prefix caches [44], [46]. Workflow predictions can initiate prefix or KV transfer before a model call, extending ideas in KVFlow and Pythia [13], [47].

Semantic and physical page sizes need not match. A semantic evidence page may compile into several KV blocks; several tiny constraints may share one canonical prefix block. The mapping table records the **ordered** semantic versions and exact prefix digest that contributed to each runtime block so that invalidation remains exact. Under ordinary causal transformers, independently cached blocks cannot be freely concatenated unless their positional and prefix dependencies match; otherwise VCM re-prefills the affected suffix. Model-native architectures may relax this restriction, but must specify the attention and position semantics that make composition sound.

VCM-Runtime additionally distinguishes an advisory cache hint from an **accepted resident-materialization claim**. A conformant claim binds a unique claim identity; source, representation, model, principal, policy, and ordered-prefix keys; a materialization predicate; a deadline or reuse horizon; and ordered outcomes such as `READY`, `EVICTED`, `INVALIDATED`, or `RESTORE_FAILED`. The runtime must either satisfy the accepted predicate or return a claim-scoped fail-closed outcome before the caller relies on reuse. Generic priority, time-to-live, offload, or routing controls are useful substrates, but they do not by themselves accept this responsibility [54]. Policy-directed span edits such as Leyline can serve as lower-level mechanisms when their position-correction and lifecycle semantics are included in the claim descriptor [53].

11.6 Learned policies inside a verified envelope

Several decisions benefit from learning:

- future page probability;
- best eligible representation for each use contract;
- co-access clustering and page size;
- likely fault cost;
- summary codec selection;
- eviction and prefetch ranking;
- when external verification is worthwhile.

However, learning operates inside deterministic constraints. The optimizer cannot bypass capabilities, omit mandatory lanes, reinterpret provenance roles, suppress a known contradiction, or consume information that was unavailable when the decision was made. Every learned retention, compression, and prefetch decision logs an observation timestamp, feature manifest, model version, and decision scope. Offline outcome labels remain separate from the online feature record so that observability leakage can be audited.

A promising direction is constrained reinforcement learning on long-horizon outcomes, where rewards include task quality, latency, evidence coverage, privacy exposure, security failures, and user corrections. Training data should include adversarial and stale-memory episodes, not only successful recall.

11.7 Open interoperability contract

For VCM to become a research substrate rather than a monolithic product, page manifests and certificates should have an open schema. Portable Agent Memory demonstrates the complementary value of provenance-verified serialization and cross-runtime re-hydration [33]; a VCM implementation should be able to mount such a package, preserve its authority ceilings, and derive local residency forms without rewriting its provenance. Interoperability requires agreement on:

- stable address and version semantics;
- provenance and execution classes;
- representation and use-contract schemas;
- capability and purpose labels;
- contradiction and supersession edges;
- invalidation and deletion events;
- model/runtime cache keys;
- telemetry for faults, promotion, and eviction;
- benchmark trace formats.

A portable page should not require trusting the sender’s summary. The receiver can verify hashes, inspect lineage, recompute representations, and apply stricter governance.

11.8 Deployment regimes

Black-box VCM compiles text around a closed model API. It can implement typed memory, representation certificates, snapshots, mounts, staging, and security, but page faults require additional calls and KV integration is limited.

Cooperative VCM controls server-side prefix caching, structured prompts, tokenizer-aware pages, and perhaps latent adapters. It can overlap I/O with tool calls and reuse stable context efficiently.

Native VCM integrates semantic fault handling, page-aware attention, model-runtime cache movement, and hardware scheduling. It offers the best latency and attribution but requires model and serving-stack changes.

The architecture should be evaluated across all three. A benefit that appears only in a hypothetical native runtime may still be scientifically interesting, but it should be distinguished from benefits available today.

11.9 Conformance profiles and the minimal core

VCM is a contract, not a requirement that every implementation deploy every optimization at once. Public claims should name the implemented profile:

- **VCM-Core:** immutable versioned addresses; typed provenance and execution classes; source-linked representations and use contracts; authority non-escalation; an authorized protected minimum set with explicit unsafe-fit behavior; semantic faults; and auditable promotion and writeback.
- **VCM-Governed:** VCM-Core plus capability-scoped mounts, taint, purpose limitation, privacy-aware views, contestability, and deletion closure.
- **VCM-Transactional:** VCM-Core plus named snapshots, atomic commits, causal metadata, copy-on-write branches, and dependency-aware invalidation.
- **VCM-Predictive:** VCM-Core plus explicit plan artifacts, calibrated demand forecasts, non-model-visible staging, cancellation, and prefetch-regret accounting.
- **VCM-Runtime:** VCM-Core plus canonical token-prefix objects, cache-key completeness, prefix/KV lifecycle integration, accepted resident-materialization claims with ordered fail-closed outcomes, and hardware-aware scheduling.

The profiles are composable. A black-box implementation can be Core, Governed, Transactional, and Predictive without claiming native KV composition. A runtime system can implement VCM-Runtime only if it also satisfies Core identity, authority, and invalidation rules. This separation makes partial implementations comparable and prevents an isolated `search_memory` function or prefix cache from being labeled VCM without the correctness contract.

12. Evaluation: VCM-Bench

A systems proposal becomes useful to research only when its claims can be falsified. VCM-Bench should evaluate not just whether an agent recalls a fact, but whether it maintains correct behavior, evidence, permissions, and latency across changing

tasks. Existing benchmarks cover complementary slices: LongMemEval and LoCoMo emphasize long-term dialogue, MemoryAgentBench and MemoryArena couple memory to incremental action, MemBench broadens memory levels and interaction modes, MemGym extends evaluation into tool use, deep research, coding, and computer interaction, and M³ Exam adds realistic multimodal memory and on-demand raw-visual access [68], [71], [72], [73], [74], [75], [76]. VCM-Bench is a composition and systems layer over such tasks, with controlled memory traces, ground-truth obligations, paging deadlines, permission views, observability snapshots, and runtime telemetry.

12.1 Research questions

The evaluation should answer eight primary questions:

- **RQ1 — Continuity:** Does VCM preserve constraints, decisions, corrections, commitments, and scoped preferences over long horizons better than competing memory methods?
- **RQ2 — Efficiency:** Does it improve task quality per active token and reduce end-to-end cost relative to long-context replay or reactive retrieval?
- **RQ3 — Latency:** Does planner-guided prefetch lower semantic page-fault stalls without excessive cache pollution or privacy exposure?
- **RQ4 — Fidelity:** Do evidence-carrying representations reduce summary drift, unsupported reconstruction, provenance-role collapse, and source-attribution errors?
- **RQ5 — Adaptation:** Does task-conditioned revaluation retrieve old but newly relevant context while avoiding irrelevant historical residue?
- **RQ6 — Safety:** Do type, taint, capability, and provenance controls reduce memory poisoning, control-flow manipulation, stale-memory use, and over-personalization?
- **RQ7 — Systems scaling:** How do benefits change with memory size, task locality, modality, page granularity, model context size, storage latency, concurrent agents, and model-runtime cache support?
- **RQ8 — Observability and complexity:** Do gains survive decision-time observability constraints and full accounting of memory construction, verification, maintenance, implementation complexity, and operator burden relative to simple typed or deterministic memory?

12.2 Hypotheses

A preregistered evaluation can test the following hypotheses:

- **H1:** At equal active-token budgets, VCM improves long-horizon task success and constraint retention over truncation, rolling summaries, and similarity-only RAG.
- **H2:** Evidence-carrying representations reduce unsupported atomic claims, provenance-role collapse, and source-attribution errors relative to unverified recursive summaries.
- **H3:** Plan-guided prefetch lowers p50 and p95 task-stall latency relative to demand-only paging when the plan has measurable predictive structure.
- **H4:** Non-model-visible staging preserves the latency benefit of prefetch while reducing irrelevant-context performance loss relative to direct speculative prompt insertion.
- **H5:** Typed retrieval channels improve correction, rejection, and decision recall beyond dense similarity at matched retrieval volume.
- **H6:** Governance invariants reduce persistent memory-attack success and excess sensitive-page exposure relative to memory systems that treat retrieved content as undifferentiated prompt text.
- **H7:** Cross-layer semantic-to-KV mapping reduces repeated prefill cost for stable mounted contexts without increasing stale-cache or cross-principal errors.
- **H8:** Observability-safe value estimation outperforms recency and uniform retention under blind future-query conditions, while any additional VCM profile is Pareto-superior to VCM-Core or a simple typed baseline only on workloads that exercise its corresponding contract.

12.3 Baselines

Evaluation should include strong and diverse baselines:

1. full native context where the complete history fits;
2. sliding-window truncation;
3. recursive rolling summary;
4. top- k lexical, dense, and hybrid RAG;
5. active or interleaved retrieval;
6. simple typed or deterministic systems such as ENGRAM and DMF, plus a source-preserving episodic baseline modeled on MemMachine [32], [34], [35];

7. persistent agent-memory systems such as MemGPT, Mem0, A-MEM, MemoryOS, MemOS, or comparable reproducible implementations;
8. learned memory-operation policies such as Agentic Memory, multi-factor value models, and observability-safe retention [64], [65];
9. semantic demand-paging and harness-managed systems such as Pichay, Neural Paging, and ClawVM where code or faithful reimplementations are available;
10. programmable context registries and compilers such as RAMPART [12];
11. provenance-tiered and typed systems such as TierMem and MemIR;
12. hierarchical, trace-compiled, portable, or agent-native memory systems such as ByteRover, CMV, VCC, and Portable Agent Memory where reproducible [33];
13. prompt-compression systems such as LongLLMLingua or RECOMP;
14. predictive retrieval and prefix/KV-aware serving systems such as Predictive Prefetching, Pythia, PBKV, TokenCake, directive-based interfaces, and fail-closed resident claims for the latency and runtime tracks [13], [19], [51], [52];
15. oracle variants with perfect page identity, future-reference knowledge, mandatory-set labeling, or summaries solely as upper bounds, never as primary deployable comparisons.

Baselines must use the same underlying model, tool set, source corpus, and active-token budget where possible. Results should separately report algorithmic gains and gains caused by a more capable base model.

12.4 Evaluation tracks

Track A: Long-term conversational continuity

Use and extend LongMemEval, LoCoMo, and related memory benchmarks [71], [72]. Add project-specific preferences, corrections, commitments, and superseded facts. Measure whether the agent recalls not merely an answer but the current valid state and appropriate interaction policy.

Track B: Incremental agent tasks

Use MemoryAgentBench, MemoryArena, MemBench, and MemGym-style interactions that require memory updates across dependent sessions and realistic tool environments [73], [74], [75], [76]. Add tool state, rejected plans, branching tasks, coding and research artifacts, computer-use traces, and delayed consequences. Report memory-isolated scores when the benchmark permits separation from base reasoning and tool competence.

Track C: Multi-project context switching

Construct workloads with several large projects, each containing decisions, source corpora, unresolved issues, and private namespaces. Switch tasks unpredictably. The complete memory should exceed the model window by orders of magnitude while each task phase has a bounded semantic working set. Measure mount latency, return-continuation quality, wrong-project contamination, and context-switch thrashing.

Track D: Evidence and exactness

Require long-form synthesis, quotations, code edits, mathematical claims, procedure execution, and cross-modal claims whose decisive evidence lives in images, diagrams, screenshots, audio, or video. Score atomic factual precision using methods related to FActScore and retrieval-grounding metrics such as RAGAS [77], [78], augmented with source-span or source-region accuracy, certificate coverage, modality-correct escalation, and exactness-fault behavior. M³Exam can seed tasks in which raw visual evidence should be loaded only when needed [68].

Track E: Temporal validity and contradiction

Insert facts that become stale, decisions that are superseded, and evidence that conflicts across sources. Evaluate whether the system distinguishes historical events from current state, identifies unresolved conflict, and revalidates when required. STALE provides a starting point for invalid-memory evaluation [62].

Track F: Personalization restraint

Create local, global, temporary, and contradictory preferences. Evaluate correct application, abstention outside scope, update behavior, and user correction. OP-Bench motivates this track [61].

Track G: Memory security and privacy

Evaluate query-only injection, poisoned experience retrieval, cross-session steering, indirect prompt injection, privilege escalation through remembered procedures, and coordinated multi-page attacks [55], [56], [57], [58]. Add protected-lane stuffing and self-pinning attacks, plus privacy probes that test whether speculative prefetch or shared caches expose sealed pages.

Track H: Systems and runtime efficiency

Replay task-plan traces under controlled storage, network, tokenizer, model, and accelerator conditions. Measure semantic fault latency, prefetch lead time, page-transfer volume, prompt tokens, prefill time, KV reuse, accelerator utilization, energy, and throughput. Compare demand-only and plan-guided policies, and evaluate semantic-to-runtime cache mapping alongside PagedAttention-, RadixAttention-, predictive-retrieval-, workflow-aware-, PBKV-, TokenCake-, directive-edit-, and fail-closed resident-claim approaches [13], [19], [44], [46], [47], [51], [52], [53], [54].

Track I: Blind retention under unknown future requests

Force consolidation and eviction decisions before the later query is sampled or revealed. Persist a cryptographic hash of the decision-time observable feature record and replay the policy from that snapshot. Compare recency, uniform retention, simple typed rules, multi-factor value, OSL-MR-style constrained optimization, and VCM policies under identical budgets [64], [65]. Report both blind retention utility and oracle upper bounds, but never mix them. Include distribution shift, delayed utility, costly reacquisition, stale evidence, and resurfacing of rarely used pages.

12.5 Canonical adversarial continuity scenario

A benchmark episode can contain the following sequence:

- early in the history, the user states a general preference for concise routine answers;
- later, the user requests exhaustive analysis for a specific research project;
- the agent proposes an incremental prototype, which the user explicitly rejects for that project;
- a trusted source supports one architecture, then a newer source changes a key fact;
- an external document contains an instruction to ignore project constraints;
- a prior assistant summary incorrectly generalizes the user’s project-specific preference;
- the final request asks for a comprehensive architecture and exact citations.

A correct system should load the project-specific exhaustive preference rather than the general concise default, preserve the rejection of prototype framing without concluding that the user always rejects prototypes, revalidate the stale source, quarantine the external instruction as data, correct the overgeneralized summary, and promote exact evidence before citation. This scenario evaluates context typing and governance, not just fact retrieval.

12.6 Metrics

Table 6 groups core metrics.

Table 6. Core VCM-Bench metric groups.

Dimension	Metrics
Task quality	success rate, calibrated judge score, tool-action correctness, long-form factual precision
Mandatory state	survival at use point, unsafe-fit rate, silent-omission rate, protected-lane completeness
Continuity	decision continuity, correction application, commitment completion, rejection recall, context-switch recovery
Context efficiency	active tokens, context value density, quality per token, bytes and tokens moved
Paging	fault rate by class, recovery success, p50/p95 latency, deadline miss rate
Prediction	prefetch precision/recall, calibration, useful lead time, cancellation cost, prefetch regret
Interference	optional-page displacement, irrelevant-context quality loss, wrong-project contamination
Residency	working-set size, eviction regret, re-fault rate, thrash index,

Dimension	Metrics
Derivation	page-granularity efficiency atomic claim coverage, negation/scope preservation, query-relative sufficiency, summary drift
Observability	future-information leakage rate, blind retention utility, oracle gap, delayed-utility regret, resurfacing recall
Evidence and authority	source-span or source-region accuracy, evidence coverage, unsupported claims, correct exactness faults, authority-escalation rate
Temporal reasoning	stale-memory use, supersession accuracy, revalidation precision and latency
Security	attack success rate, poisoned-page promotion, control-flow influence, unauthorized tool justification
Privacy	excess pages accessed, sensitive bytes staged, access-pattern leakage, unauthorized exposure, purge latency
Consistency	mixed-version errors, read-your-writes failures, stale-cache use, merge-conflict loss
Runtime	prefill time, accepted-claim success/failure, KV hit rate, HBM/DRAM/SSD traffic, throughput, energy, cost per successful task
Complexity	offline build time, index/verification maintenance, storage amplification, configuration surface, failure-recovery labor, reproducibility

Mandatory-state survival is measured at each use point, not only at the end of an episode. A system fails this metric if a required correction or constraint is absent when the relevant action is chosen, even if it can answer a later recall question. **Unsafe-fit rate** distinguishes honest capacity failures from silent omission.

Prefetch precision counts a page as useful only when it is promoted and causally contributes before its deadline. Merely being semantically related is insufficient. **Prefetch regret** adds wasted fetch/materialization/exposure cost to avoidable fault cost from missed pages. **Attention-interference penalty** is estimated with paired replays that remove optional pages while holding mandatory state fixed. **Thrash index** combines repeated re-faults, residency turnover, transfer time, and lack of task progress.

Evidence coverage is the fraction of atomic output claims supported by loaded evidence pages whose certificates permit that use; it is stratified by claim risk. **Authority-escalation rate** counts cases in which a representation affects planning, tool use, or assertions beyond its source and policy ceiling. **Deletion closure** measures whether a deleted fact remains recoverable through summaries, embeddings, logs, shared pages, staging, or runtime caches after the stated purge deadline.

Future-information leakage rate is the fraction of online decisions whose logged feature manifest depends directly or indirectly on a later query, answer, gold evidence label, or benchmark-only dependency annotation. **Oracle gap** reports the difference between a deployable blind policy and a future-aware upper bound. Complexity metrics are reported alongside quality because a many-service architecture that only ties a three-type retriever is not a research or engineering win.

12.7 Ablation studies

Ablations should remove one architectural feature at a time:

- scalar rather than vector importance;
- decision-time observable valuation versus future-query-leaked valuation;
- no protected lanes;
- protected lanes without explicit unsafe-fit handling;
- linear representation ladder rather than use contracts;
- no authority non-escalation;
- no resurfacing budget;
- dense similarity only;
- simple three-type retrieval or deterministic scoring rather than the full VCM policy stack;
- no rejection ledger;
- no contradiction graph;
- no temporal validity;

- no representation certificates;
- recursive summary without raw anchors;
- textual derivatives without modality-native source fallback;
- demand paging without plan prefetch;
- prefetch directly into model-visible context;
- no staging taint gate;
- LRU rather than semantic replacement;
- no task snapshots;
- no capability-based mounts;
- no semantic-to-KV mapping;
- advisory cache hints instead of accepted fail-closed resident claims;
- no counterfactual utility feedback.

Interactions matter. For example, prefetch may look harmful without staging, while staging may appear unnecessary without adversarial or low-precision forecasts. Factorial experiments should identify such dependencies.

12.8 Experimental protocol

A rigorous protocol should:

- preregister hypotheses and primary metrics;
- release task traces, memory events, page manifests, and ground-truth dependency graphs where privacy permits;
- fix active-token budgets and separately sweep them;
- test several model families and context lengths;
- report confidence intervals, effect sizes, and paired comparisons across identical episodes;
- tune page boundaries, retrieval channels, and learned policies only on disjoint development traces;
- freeze and hash the observable feature record at every retention, compression, and prefetch decision; execute blind policies before revealing future queries or gold dependencies;
- report oracle policies only as clearly labeled upper bounds and quantify the oracle gap;
- ensure page boundaries, manifests, dependency graphs, and benchmark metadata available online do not encode future answers;
- separate offline memory construction, verification, indexing, maintenance, and operator-recovery cost from online latency, while also reporting amortized total cost;
- include cold-cache, warm-cache, context-switch, and mandatory-set-overflow conditions;
- evaluate under multiple storage and network latencies;
- report prefetch calibration and regret, not only hit rate;
- run paired interference trials with optional pages inserted and removed;
- validate conformance invariants independently of end-task quality;
- test benign, stale, contradictory, and adversarial histories;
- report governance denials and clarification requests as outcomes rather than hiding them;
- audit judge-model bias with human and deterministic checks;
- publish failure traces, not only averages;
- disclose benchmark-specific annotations, oracle information, architecture-specific supervision, and all external model calls used to create or maintain memory so that answer leakage and hidden compute are distinguishable from memory management;
- publish a minimal VCM-Core configuration and a strong simple typed/deterministic baseline before evaluating optional predictive, transactional, governed, or runtime profiles.

The architecture should be considered successful only if it improves meaningful long-horizon outcomes at acceptable overhead and does not obtain efficiency by weakening evidence, privacy, or security guarantees.

12.9 Decision rules and falsification criteria

VCM should be reported as a Pareto frontier rather than collapsed into one favorable weighted score. At minimum, experiments should identify task quality, mandatory-state survival, evidence fidelity, online latency, total compute, privacy exposure, and attack success. A configuration is dominated if another achieves at least the same quality and safety with no greater resource cost.

The paper’s central claims would be weakened or falsified under any of the following outcomes:

- VCM-Core does not improve mandatory-state survival or evidence fidelity over strong typed-memory and provenance-tiered baselines at matched active-token and model budgets;
- predictive staging fails to reduce p95 stall latency in workloads whose future references are demonstrably predictable, or its avoided stalls are outweighed by prefetch regret and excess sensitive-page access;
- representation certificates do not reduce unsupported reconstruction or role collapse compared with source-linked baselines;
- protected compilation improves constraint retention only by causing unacceptable refusal, unsafe-fit, or task-decomposition rates;
- semantic-to-runtime mapping offers no end-to-end benefit after accounting for prefill, transfer, invalidation, and recomputation costs;
- benefits disappear once offline memory-construction, verification, index-maintenance, configuration, and failure-recovery costs are charged;
- the full architecture does not outperform VCM-Core, ENGRAM/DMF-style simple baselines, or source-preserving episodic retrieval on workloads that supposedly require its additional contracts;
- gains occur only on benchmark traces whose future queries, dependency annotations, page boundaries, summaries, or feature pipelines leak the answer.

Conversely, a negative result in a low-locality or globally coupled workload would support the paper’s stated boundary rather than refute all of VCM. The decisive test is whether the architecture predicts **where** paging helps, **where** it does not, and which mechanism is responsible.

13. Discussion

13.1 From retrieval to residency

Retrieval asks, “Which chunks are similar to the current query?” VCM asks a broader question: “Which verified semantic objects should be resident, at what fidelity, for how long, under which authority, given the task’s likely future?” Retrieval remains a mechanism inside the system, but residency adds planning, lifecycle, and resource semantics.

This distinction matters because critical context is often behaviorally important rather than lexically salient. A recent correction, a hard prohibition, or a rejected design may share few words with the current request. A context compiler retrieves by role and dependency, then decides whether to keep the page resident across several model calls.

13.2 From transcript to state

A transcript is an event record. Agent state is a typed interpretation of events. Conflating them produces two opposite errors: replaying too much irrelevant history, or compressing away the structure needed for correct behavior. VCM preserves both levels. The immutable event substrate supports audit and re-derivation; the context ledger supports efficient task state.

This resembles the distinction between a database log and its materialized views. A view is useful because it is structured for a query, but it must be invalidated when sources change and cannot be treated as the sole ground truth.

13.3 From larger windows to effective working context

Native context growth remains valuable. VCM does not oppose it. A larger window raises the maximum feasible working-set size and reduces fault frequency. VCM improves how that capacity is allocated. The combined system can use a large native window for globally coupled phases and virtualize the much larger history surrounding them.

The critical scientific question is not whether VCM can “simulate infinite context,” but how quality scales as the ratio of durable memory to active window grows, and how that scaling depends on semantic locality, plan predictability, compression fidelity, and page-fault cost.

13.4 Context allocation as a constrained market

The context compiler can be viewed as a market in which pages bid for scarce active capacity. This metaphor highlights opportunity cost: loading one page displaces another. Yet a pure market is unsafe because mandatory policy and constraints could be outbid. VCM therefore combines protected constitutional lanes with marginal-value allocation in the discretionary remainder.

This framework may enable useful diagnostics. A page with high token cost and low marginal utility should be compressed or evicted. A page with low immediate utility but high catastrophic-loss prevention should be pinned. A privacy-sensitive page may have high task value but still be denied because exposure cost dominates.

13.5 Memory as a control-flow substrate

Persistent memory is not passive. It shapes future planning, retrieval, and tool use. Memory systems must therefore be analyzed like control-flow systems. A poisoned page that changes which tools are called or which evidence is trusted can have effects far beyond an incorrect answer. This motivates execution classes, capability checks, taint propagation, and independent tool authorization.

Security evaluation should measure not only whether an attack string is repeated, but whether it changes plan topology, page promotion, permission requests, or durable memory writes.

13.6 Evidence-carrying memory and epistemic humility

Evidence-carrying representations address a structural problem: once a summary is fluent, future models may treat it as established truth. A certificate keeps source integrity, evidence support, interpretation, uncertainty, and execution authority separate. A low-cost synthesis can guide routing while a high-stakes answer faults to exact evidence.

The paper deliberately avoids claiming that natural-language summaries carry formal proofs of truth. A representation certificate can prove or attest to machine-checkable facts—hashes, source lineage, transformation identity, verifier execution, policy decisions, and permitted uses. The substantive claim can still be uncertain, disputed, or source-relative. This distinction is necessary for scientific and security precision.

13.7 Open research problems

Several problems remain open.

Future-query modeling. How should a compressor define the query family for which a representation is sufficient, especially before future tasks are known?

Learned semantic page boundaries. How can pages adapt to co-access patterns without destroying human-understandable identity and auditability?

Faithful latent representations. Can learned latent pages provide large compression gains while retaining interpretable certificates and reliable fault paths?

Plan calibration. How should a planner express uncertainty, and how should prefetch policies remain robust when the plan changes rapidly?

Causal memory utility. Can the system estimate whether a page caused a better outcome rather than merely being present?

Global reasoning detection. Can an agent recognize when paging will be inadequate because the task requires simultaneous attention to a large set?

Multi-model coherence. How should pages and caches remain valid across models with different tokenizers, capabilities, or interpretations?

Privacy-preserving addressing. Can a root reveal that relevant memory exists without leaking sensitive namespace structure or access patterns?

Memory policy verification. Which governance invariants can be formally verified across language-model, database, and serving-stack boundaries?

Forgetting and unlearning. How can deletion closure include learned adapters or models influenced by memory without retraining the entire system?

Human mental models. What interface lets users understand mounts, scopes, summaries, and deletion without exposing overwhelming implementation detail?

Benchmark realism. How can long-term memory be evaluated over months or years without relying exclusively on synthetic histories or expensive human studies?

13.8 Broader implications

If VCM-like systems succeed, agents could maintain durable project continuity without continuously replaying private history; resume complex work from source-bound checkpoints; switch among many contexts with bounded latency; share read-only evidence while preserving private annotations; and audit how remembered information influenced an action. The same mechanisms could improve coding agents, research assistants, enterprise workflows, tutoring systems, and multi-agent coordination.

The risks are equally broad. A high-fidelity memory system can create deeper surveillance, more persistent manipulation, and stronger lock-in if users cannot inspect or control it. Memory quality cannot be separated from memory governance. A technically effective VCM that maximizes recall while minimizing user agency would be a research failure.

14. Conclusion

Long-horizon language-model agents need more than larger context windows, vector retrieval, or rolling summaries. They need a memory hierarchy that distinguishes durable state from active context and treats selection, compression, timing, consistency, provenance, security, and privacy as one systems problem.

Virtual Context Memory proposes such a hierarchy. It stores interaction and task state as typed, versioned semantic pages in a durable context ledger; exposes them through stable mountable addresses; creates multiple certified representations; forecasts demand from explicit plans; stages and promotes pages under governance; compiles a protected working set into the finite context window; and links semantic memory to model-runtime prefix and KV caches without confusing cache state for truth. It preserves a recoverable path from compressed context to evidence, represents contradictions and rejected branches directly, and uses transactional snapshots to keep long-running tasks coherent.

The central reframing is simple:

A long-horizon agent should not carry all of its history. It should maintain a governed virtual context space and make the right verified working set resident at the right time.

VCM does not make context infinite, and it does not make lossy summaries reversible. It makes durable context addressable, use-constrained, stageable, faultable, governable, and coherently compilable. Whether this contract improves real agents is an empirical question. The explicit novelty boundary, formal limitations, ABI, invariants, failure responses, and VCM-Bench agenda are intended to make that question answerable—and to make negative results informative.

Appendix A. Canonical Semantic Page Schema

The following illustrative schema shows the fields required for a portable page. Implementations may use binary or normalized storage, but the semantic contract should remain explicit.

page:

address: `vcm://principal/project/object-id@version`

content_hash: `sha256:...`

immutable_version: `17`

alias_history:

- alias: `vcm://principal/project/object-id@latest`

resolved_at: `2026-06-18T20:00:00Z`

resolver_signature: ...

type: `decision`

execution_class: `authorized_task_state`

authority_ceiling:

behavioral: `task_scoped_instruction`

evidential: explicit_user_statement
action: none_without_live_capability
status: active
subject: project-x
scope:
 task: architecture-paper
 domain: research
 temporal:
 valid_from: 2026-06-18T00:00:00Z
 valid_until: null

authoritative_sources:
 - event_hash: sha256:...
 source_role: explicit_user_statement
 modality: text
 spans:
 - start: 114
 end: 392
 trust: high

claims:
 - id: claim-1
 text: "The current project should target the final architecture rather than an incremental prototype."
 support:
 - event_hash: sha256:...
 span: [114, 392]
 certainty: explicitly_stated

relations:
 depends_on: []
 supports: []
 contradicts: []
 supersedes: []
 rejected_because: []
 invalidates: []

valuation:
 decision_time: 2026-06-18T20:12:09Z
 observable_feature_manifest: sha256:...
 policy_version: importance-policy-v3
 future_query_visible: false
 offline_outcome_label: stored_separately

importance:
 task_relevance: 0.98
 failure_prevention: 0.91
 decision_weight: 0.95

future_utility: 0.72
stability: 0.74
reuse_probability: 0.68

risk:

staleness: 0.05
contradiction: 0.10
privacy_sensitivity: 0.12
poisoning: 0.03
compression_loss: 0.20
context_pollution: 0.04

governance:

owner: principal:user-123
capabilities_required:
 read: [project-x-member]
 write: [project-x-owner]
allowed_purposes: [research, drafting]
prefetch_policy: allowed_to_encrypted_staging
sharing: private
retention: until_project_deletion
deletion_key_id: key-7f9a

representations:

routing_capsule:
 object_hash: sha256:...
 token_estimate: 31
 use_contracts_supported: [routing, prefetch]
 forbidden_uses: [quotation, tool_authorization]
 source_fallback: vcm://principal/project/object-id@17#raw
 representation_certificate: cert:routing-17

structured_synthesis:

 object_hash: sha256:...
 token_estimate: 146
 use_contracts_supported: [planning, drafting]
 forbidden_uses: [exact_quotation]
 declared_losses: [verbatim_wording, unrelated_episode_detail]
 source_fallback: vcm://principal/project/object-id@17#raw
 representation_certificate: cert:synthesis-17

exact_excerpts:

 object_hash: sha256:...
 token_estimate: 278
 use_contracts_supported: [exact_wording, dispute_resolution]
 representation_certificate: cert:excerpts-17

residency:

 task_snapshot: snap:project-x-93
 locations:


```

- tier: dram_compressed
  representation: structured_synthesis
- tier: object_store
  representation: raw_source
pinned_until: null
last_accessed: 2026-06-18T20:12:09Z

audit:
  created_by: memory-ingestor-v4
  verified_by: [scope-checker-v2, claim-span-checker-v3]
  created_at: 2026-06-18T20:00:01Z
  last_revalued: 2026-06-18T20:12:09Z

```

Appendix B. Reference Algorithms

B.1 Compile working context

```

COMPILE_CONTEXT(task, snapshot, budget):
  packet <- empty role-labeled packet

  packet.add(current_system_policy(snapshot), lane=POLICY)
  packet.add(current_user_request(task), lane=REQUEST)
  packet.add(task.objective_and_plan_frontier(), lane=TASK)

  mandatory <- union(
    active_constraints(task, snapshot),
    recent_corrections(task, snapshot),
    unresolved_commitments(task, snapshot),
    required_procedures(task.next_actions),
    evidence_obligations(task.planned_claims)
  )

  mandatory_reps <- []
  for obligation in mandatory:
    rep <- resolve_cheapest_eligible_representation(
      obligation.page, obligation.use_contract, snapshot)
    rep <- GOVERN_AND_FAULT_IF_NEEDED(rep, task, snapshot)
    mandatory_reps.append((obligation, rep))

  if not fits_all_resource_budgets(packet, mandatory_reps, budget):
    return UNSAFE_FIT(mandatory_reps, budget)

  for obligation, rep in mandatory_reps:
    packet.add(rep, lane=obligation.protected_lane)

  candidates <- multi_channel_retrieve(
    semantic=task.query,
    dependencies=task.plan_frontier,
    decisions=task.active_decision_scope,
    corrections=task.entities,

```

```

    contradictions=packet.claims,
    rejections=task.proposed_alternatives,
    temporal=task.freshness_requirements,
    preferences=task.interaction_scope,
    predictions=task.context_demand_forecast
  )

  candidates <- dependency_and_conflict_closure(candidates, snapshot)
  candidates <- remove_denied_or_invalid(candidates, task)
  observable <- freeze_observable_features(task, snapshot, now())
  bids <- estimate_marginal_value(candidates, packet, observable, budget)
  assert no_future_information_dependency(bids, observable)

  while packet.has_discretionary_capacity():
    page, rep <- highest_positive_feasible_bid(bids)
    if none: break
    packet.add(rep, lane=page.role_lane)
    update_bids_for_redundancy_and_dependencies(bids, packet)

  context_map <- context_map_for_nearby_handles(task, packet)
  if context_map.fits_remaining_budget() and context_map.passes_governance():
    packet.add(context_map, lane=MAP)
  return verify_packet_invariants(packet, task, snapshot, budget)

```

B.2 Planner-guided prefetch

```

PREFETCH_FROM_PLAN(plan, snapshot, resource_budget):
  forecast <- derive_context_demand(plan)

  for demand in forecast.sorted_by_deadline():
    page <- resolve_address(demand.address, snapshot)
    if not capability_allows_prefetch(page, demand.purpose):
      continue

    expected_value <- demand.probability *
page.avoidable_fault_latency(demand.use_contract)
    expected_cost <- fetch_materialize_cost(page, demand.use_contract)
    expected_cost += pollution_cost(page, plan)
    expected_cost += privacy_exposure_cost(page, demand.purpose)

    if expected_value <= expected_cost:
      continue

    token <- reserve_staging_capacity(page, demand.deadline)
    async_fetch_and_materialize(page, demand.use_contract, token)
    mark_non_model_visible(page, token)

  on_plan_frontier_change:
    cancel_losing_branch_fetches()
    purge_disallowed_staging_pages()
    promote_ready_pages_that_pass_all_gates()

```

B.3 Transactional memory commit

```

COMMIT_MEMORY(transaction, task_snapshot):
  assert transaction.sources_are_immutable()
  typed_cells <- segment_and_type(transaction.events)
  provisional_pages <- build_or_update_pages(typed_cells, task_snapshot)
  relations <- detect_support_conflict_supersession(provisional_pages)
  representations <- generate_required_representations(provisional_pages)
  certificates <- verify_and_certify(representations)
  governance <- classify_and_authorize(provisional_pages, certificates)

  if any hard verification or governance failure:
    rollback_to_private_branch(transaction)
    return FAILURE

  invalidation_set <- trace_affected_descendants(provisional_pages, relations)

  atomic_write(
    page_versions=provisional_pages,
    graph_relations=relations,
    certificates=certificates,
    governance=governance,
    invalidations=invalidation_set,
    audit=transaction.audit_record
  )

  asynchronously_refresh_indexes()
  purge_or_mark_stale_runtime_caches(invalidation_set)
  return COMMITTED_VERSION_SET

```

B.4 Observability-safe revaluation

```

REVALUE_FOR_RETENTION(page, event_stream, policy, decision_time):
  observable <- snapshot_features(event_stream, cutoff=decision_time)
  manifest_hash <- hash(feature_names_and_values(observable))

  assert observable.excludes(
    future_queries,
    future_answers,
    gold_evidence_labels,
    future_dependency_annotations,
    post_decision_page_boundaries
  )

  value, uncertainty <- policy.score(page, observable)
  decision <- constrained_retention_action(
    value=value,
    uncertainty=uncertainty,
    budget=current_retention_budget(),
    governance=page.governance
  )

```

```

log_online_decision(
    page=page.address,
    decision_time=decision_time,
    feature_manifest=manifest_hash,
    policy_version=policy.version,
    decision=decision
)
return decision

```

```

AFTER_OUTCOME(outcome):
    attach_offline_label_to_separate_training_record(outcome)
    never_mutate_the_original_online_feature_manifest()

```

Appendix C. VCM Invariants Checklist

A conforming implementation should be testable against the following checklist:

1. Every model-visible durable memory span resolves to an immutable page version and provenance role.
2. Every derived page representation exposes a certificate, authority ceiling, declared loss, and fallback path.
3. Every task uses a named snapshot and receives read-your-writes behavior.
4. Every conflict is represented, adjudicated, or explicitly excluded with a reason.
5. Every prefetch enters non-model-visible staging before promotion.
6. Every promotion passes capability, purpose, taint, freshness, use-contract, and authority gates.
7. Every tool action receives current authorization independent of memory text.
8. Every runtime cache is keyed by source and representation versions, model, tokenizer, role layout, policy, principal, redaction view, permissions, and task snapshot as required.
9. Every page deletion initiates graph-based descendant and cache handling.
10. Every user preference records scope, evidence mode, confidence, and correction controls.
11. Every context switch checkpoints dirty state and mounts a versioned root.
12. Every page-fault denial or failure produces a safe fallback rather than unsupported reconstruction.
13. No transformation or cache materialization increases behavioral authority beyond source and policy ceilings.
14. If the protected minimum set cannot fit, compilation returns an explicit unsafe-fit result rather than dropping a mandatory page.
15. Semantic materialization from a lossy derivative creates a new certified representation; it is never labeled exact decompression.
16. Learned scoring, compression, and prefetch policies operate inside deterministic capability, authority, snapshot, and deletion constraints.
17. Low retrieval frequency alone cannot trigger permanent deletion; resurfacing and retention are governed separately.
18. A page cannot self-designate as mandatory, pinned, trusted, or privileged; protected admission is issuer- and policy-authorized.
19. A reusable prefix or KV object is valid only under its complete ordered-prefix, position, model, tokenizer, adapter, policy, principal, permission, redaction, and snapshot key.
20. Every online retention, compression, and prefetch decision can be replayed from a decision-time observable feature manifest that excludes future requests and oracle annotations.
21. A runtime future-reuse promise is conformant only after an identified resident-materialization claim is accepted and later produces an ordered success, invalidation, eviction, or fail-closed restoration outcome.

Appendix D. Glossary

Active context: The finite model-visible representation used for one inference step or tightly coupled group of steps.

CMMU: Context Memory Management Unit; the resolver, fault handler, residency manager, and prefetch controller for semantic pages.

C-TLB: Context translation lookaside buffer; a cache mapping semantic addresses and task snapshots to valid resident representations or denial states.

Context cell: An atomic typed memory unit such as a constraint, claim, decision, correction, or event.

Context compiler: The component that admits protected state, selects eligible optional representations, resolves conflicts and dependencies, labels provenance, and packs a working context under multiple budgets.

Materialization: Construction or retrieval of a representation that satisfies a use contract. Unlike exact decompression, semantic materialization may be lossy or generative and therefore requires a new certificate.

Observability-safe valuation: Retention, compression, or prefetch scoring that uses only features available at the decision time; future outcomes may supervise offline learning but cannot enter the online decision record.

Protected minimum set: Policy, task, constraints, corrections, commitments, procedures, and evidence obligations that must fit before discretionary context is selected.

Representation certificate: Machine-auditable lineage, loss, authority, validity, verification, and permitted-use metadata attached to a derived representation.

Resident-materialization claim: A runtime-accepted obligation binding a specific future prefix or KV materialization to complete identity keys, a readiness predicate, a horizon, and ordered fail-closed lifecycle outcomes.

Unsafe fit: A compiler result indicating that mandatory context cannot be represented within available capacity without violating an invariant.

Use contract: The query/operation family, coverage, exactness, evidence, authority, freshness, purpose, capability, and deadline requirements a representation must satisfy.

Virtual Context ABI: The inter-layer contract for addressing, mounting, resolving, staging, promoting, faulting, snapshotting, committing, invalidating, and unmounting semantic pages.

Context ledger: The versioned graph of semantic pages, provenance, dependencies, conflicts, and lifecycle events.

Context root: A mountable manifest defining a coherent namespace and entry point, such as a project or corpus.

Declared-loss derivation: A lossy transformation that explicitly identifies potential omissions and restricts downstream use accordingly.

Effective working context: The set of pages that can be made sufficiently available within task deadlines and reliability bounds, not merely the contents of storage.

Authority vector: Separate ceilings for behavioral influence, evidential use, and action authorization, interpreted under an implementation-defined partial order.

Execution class: The authority category that determines whether content may influence behavior, serve as evidence, or remain data-only.

Rejection ledger: Structured memory of rejected, obsolete, disproven, or unsafe branches and the reasons for their status.

Semantic page: The smallest independently addressable, loadable, and governed bundle of context cells.

Semantic page fault: A request for a missing identity, detail, evidence, exactness, freshness, capability, integrity, or dependency state.

Staging cache: A non-model-visible area in which predicted pages can be fetched and materialized before relevance and governance checks permit promotion.

Task snapshot: A coherent versioned view of pages, mounts, permissions, and policies used by a running task.

Virtual context address: A stable logical identifier for a semantic page, independent of physical storage and representation.

References

- [1] N. F. Liu *et al.*, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: [10.1162/tac1_a_00638](https://doi.org/10.1162/tac1_a_00638).
- [2] C.-P. Hsieh *et al.*, “RULER: What’s the real context size of your long-context language models?” *arXiv preprint arXiv:2404.06654*, 2024, doi: [10.48550/arXiv.2404.06654](https://doi.org/10.48550/arXiv.2404.06654).
- [3] Y. Bai *et al.*, “LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks,” in *Proceedings of the 63rd annual meeting of the association for computational linguistics*, 2025, pp. 3639–3664. doi: [10.48550/arXiv.2412.15204](https://doi.org/10.48550/arXiv.2412.15204).
- [4] Y. Du *et al.*, “Context length alone hurts LLM performance despite perfect retrieval,” in *Findings of the association for computational linguistics: EMNLP 2025*, 2025, pp. 23281–23298. doi: [10.18653/v1/2025.findings-emnlp.1264](https://doi.org/10.18653/v1/2025.findings-emnlp.1264).
- [5] C. Packer *et al.*, “MemGPT: Towards LLMs as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023, doi: [10.48550/arXiv.2310.08560](https://doi.org/10.48550/arXiv.2310.08560).
- [6] T. Mason, “The missing memory hierarchy: Demand paging for LLM context windows,” *arXiv preprint arXiv:2603.09023*, 2026, doi: [10.48550/arXiv.2603.09023](https://doi.org/10.48550/arXiv.2603.09023).
- [7] L. Chen and Q. Liu, “Neural paging: Learning context management policies for turing-complete agents,” *arXiv preprint arXiv:2603.02228*, 2026, doi: [10.48550/arXiv.2603.02228](https://doi.org/10.48550/arXiv.2603.02228).
- [8] M. Rafique and L. Bindschaedler, “ClawVM: Harness-managed virtual memory for stateful tool-using LLM agents,” in *Proceedings of the sixth european workshop on machine learning and systems (EuroMLSys ’26)*, 2026, pp. 1–12. doi: [10.1145/3805621.3807648](https://doi.org/10.1145/3805621.3807648).
- [9] Q. Zhu, S. Chen, R. Yu, Z. Wu, and B. Wang, “From lossy to verified: A provenance-aware tiered memory for agents,” *arXiv preprint arXiv:2602.17913*, 2026, doi: [10.48550/arXiv.2602.17913](https://doi.org/10.48550/arXiv.2602.17913).
- [10] Z. Jin, B. Wang, J. Li, R. Xu, and M. Zhang, “Mitigating provenance-role collapse in long-term agents via typed memory representation,” *arXiv preprint arXiv:2605.25869*, 2026, doi: [10.48550/arXiv.2605.25869](https://doi.org/10.48550/arXiv.2605.25869).
- [11] C. Ouyang and R. Hou, “MemLineage: Lineage-guided enforcement for LLM agent memory,” *arXiv preprint arXiv:2605.14421*, 2026, doi: [10.48550/arXiv.2605.14421](https://doi.org/10.48550/arXiv.2605.14421).
- [12] N. Tomczak, “RAMPART: Registry-based agentic memory with priority-aware runtime transformation,” *arXiv preprint arXiv:2606.04628*, 2026, doi: [10.48550/arXiv.2606.04628](https://doi.org/10.48550/arXiv.2606.04628).
- [13] S. Yu *et al.*, “Pythia: Toward predictability-driven agent-native LLM serving,” *arXiv preprint arXiv:2604.25899*, 2026, doi: [10.48550/arXiv.2604.25899](https://doi.org/10.48550/arXiv.2604.25899).
- [14] H. Lin, H. Pao, S. Zhan, and H.-T. Zheng, “Model-native computing architecture: Envisioning future system architecture through the lens of computer architecture,” *arXiv preprint arXiv:2606.00288*, 2026, doi: [10.48550/arXiv.2606.00288](https://doi.org/10.48550/arXiv.2606.00288).
- [15] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in neural information processing systems*, 2020, pp. 9459–9474.
- [16] Z. Jiang *et al.*, “Active retrieval augmented generation,” *arXiv preprint arXiv:2305.06983*, 2023, doi: [10.48550/arXiv.2305.06983](https://doi.org/10.48550/arXiv.2305.06983).
- [17] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023, doi: [10.48550/arXiv.2310.11511](https://doi.org/10.48550/arXiv.2310.11511).
- [18] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” *arXiv preprint arXiv:2212.10509*, 2022, doi: [10.48550/arXiv.2212.10509](https://doi.org/10.48550/arXiv.2212.10509).
- [19] W. Zhang and S. Pei, “Predictive prefetching for retrieval-augmented generation,” *arXiv preprint arXiv:2605.17989*, 2026, doi: [10.48550/arXiv.2605.17989](https://doi.org/10.48550/arXiv.2605.17989).
- [20] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” *arXiv preprint arXiv:2304.03442*, 2023, doi: [10.48550/arXiv.2304.03442](https://doi.org/10.48550/arXiv.2304.03442).
- [21] J. Kang, M. Ji, Z. Zhao, and T. Bai, “Memory OS of AI agent,” *arXiv preprint arXiv:2506.06326*, 2025, doi: [10.48550/arXiv.2506.06326](https://doi.org/10.48550/arXiv.2506.06326).
- [22] Z. Li *et al.*, “MemOS: A memory OS for AI system,” *arXiv preprint arXiv:2507.03724*, 2025, doi: [10.48550/arXiv.2507.03724](https://doi.org/10.48550/arXiv.2507.03724).
- [23] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, “Mem0: Building production-ready AI agents with scalable long-term memory,” *arXiv preprint arXiv:2504.19413*, 2025, doi: [10.48550/arXiv.2504.19413](https://doi.org/10.48550/arXiv.2504.19413).
- [24] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, “A-MEM: Agentic memory for LLM agents,” *arXiv preprint arXiv:2502.12110*, 2025, doi: [10.48550/arXiv.2502.12110](https://doi.org/10.48550/arXiv.2502.12110).
- [25] J. Liu *et al.*, “SimpleMem: Efficient lifelong memory for LLM agents,” *arXiv preprint arXiv:2601.02553*, 2026, doi: [10.48550/arXiv.2601.02553](https://doi.org/10.48550/arXiv.2601.02553).
- [26] Y. Yu *et al.*, “Agentic memory: Learning unified long-term and short-term memory management for large language model agents,” *arXiv preprint arXiv:2601.01885*, 2026, doi: [10.48550/arXiv.2601.01885](https://doi.org/10.48550/arXiv.2601.01885).
- [27] M. Hu, T. Chen, Q. Chen, Y. Mu, W. Shao, and P. Luo, “HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model,” *arXiv preprint arXiv:2408.09559*, 2024, doi: [10.48550/arXiv.2408.09559](https://doi.org/10.48550/arXiv.2408.09559).
- [28] H. Yu *et al.*, “MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent,” *arXiv preprint arXiv:2507.02259*, 2025, doi: [10.48550/arXiv.2507.02259](https://doi.org/10.48550/arXiv.2507.02259).
- [29] A. Nguyen *et al.*, “ByteRover: Agent-native memory through LLM-curated hierarchical context,” *arXiv preprint arXiv:2604.01599*, 2026, doi: [10.48550/arXiv.2604.01599](https://doi.org/10.48550/arXiv.2604.01599).
- [30] C. Santoni, “Contextual memory virtualisation: DAG-based state management and structurally lossless trimming for LLM agents,” *arXiv preprint arXiv:2602.22402*, 2026, doi: [10.48550/arXiv.2602.22402](https://doi.org/10.48550/arXiv.2602.22402).
- [31] L. Zhang and M. Agrawala, “View-oriented conversation compiler for agent trace analysis,” *arXiv preprint arXiv:2603.29678*, 2026, doi: [10.48550/arXiv.2603.29678](https://doi.org/10.48550/arXiv.2603.29678).
- [32] S. Wang *et al.*, “MemMachine: A ground-truth-preserving memory system for personalized AI agents,” *arXiv preprint arXiv:2604.04853*, 2026, doi: [10.48550/arXiv.2604.04853](https://doi.org/10.48550/arXiv.2604.04853).
- [33] S. K. Ravindran, “Portable agent memory: A protocol for provenance-verified memory transfer across heterogeneous LLM agents,” *arXiv preprint arXiv:2605.11032*, 2026, doi: [10.48550/arXiv.2605.11032](https://doi.org/10.48550/arXiv.2605.11032).
- [34] D. Patel and S. Patel, “ENGRAM: Effective, lightweight memory orchestration for conversational agents,” *arXiv preprint arXiv:2511.12960*, 2025, doi: [10.48550/arXiv.2511.12960](https://doi.org/10.48550/arXiv.2511.12960).
- [35] M. Stabile and E. Zimuel, “DMF: A deterministic memory framework for conversational AI agents,” *arXiv preprint arXiv:2606.03463*, 2026, doi: [10.48550/arXiv.2606.03463](https://doi.org/10.48550/arXiv.2606.03463).

- [36] Y. Hu *et al.*, “Memory in the age of AI agents,” *arXiv preprint arXiv:2512.13564*, 2025, doi: [10.48550/arXiv.2512.13564](https://doi.org/10.48550/arXiv.2512.13564).
- [37] N. Vispute, “Reasoning provenance for autonomous AI agents: Structured behavioral analytics beyond state checkpoints and execution traces,” *arXiv preprint arXiv:2603.21692*, 2026, doi: [10.48550/arXiv.2603.21692](https://doi.org/10.48550/arXiv.2603.21692).
- [38] H. Xu, “Mesh memory protocol: Semantic infrastructure for multi-agent LLM systems,” *arXiv preprint arXiv:2604.19540*, 2026, doi: [10.48550/arXiv.2604.19540](https://doi.org/10.48550/arXiv.2604.19540).
- [39] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 13358–13376. doi: [10.18653/v1/2023.emnlp-main.825](https://doi.org/10.18653/v1/2023.emnlp-main.825).
- [40] H. Jiang *et al.*, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” *arXiv preprint arXiv:2310.06839*, 2023, doi: [10.48550/arXiv.2310.06839](https://doi.org/10.48550/arXiv.2310.06839).
- [41] J. Mu, X. L. Li, and N. Goodman, “Learning to compress prompts with gist tokens,” *arXiv preprint arXiv:2304.08467*, 2023, doi: [10.48550/arXiv.2304.08467](https://doi.org/10.48550/arXiv.2304.08467).
- [42] F. Xu, W. Shi, and E. Choi, “RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation,” in *The twelfth international conference on learning representations*, 2024. doi: [10.48550/arXiv.2310.04408](https://doi.org/10.48550/arXiv.2310.04408).
- [43] N. Verma, “Active context compression: Autonomous memory management in LLM agents,” *arXiv preprint arXiv:2601.07190*, 2026, doi: [10.48550/arXiv.2601.07190](https://doi.org/10.48550/arXiv.2601.07190).
- [44] W. Kwon *et al.*, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th ACM symposium on operating systems principles*, 2023, pp. 611–626. doi: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).
- [45] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, “vAttention: Dynamic memory management for serving LLMs without PagedAttention,” *arXiv preprint arXiv:2405.04437*, 2024, doi: [10.48550/arXiv.2405.04437](https://doi.org/10.48550/arXiv.2405.04437).
- [46] L. Zheng *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Advances in neural information processing systems*, 2024. doi: [10.48550/arXiv.2312.07104](https://doi.org/10.48550/arXiv.2312.07104).
- [47] Z. Pan *et al.*, “KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows,” in *Advances in neural information processing systems*, 2025. doi: [10.48550/arXiv.2507.07400](https://doi.org/10.48550/arXiv.2507.07400).
- [48] Z. Zhang *et al.*, “H2O: Heavy-hitter oracle for efficient generative inference of large language models,” *arXiv preprint arXiv:2306.14048*, 2023, doi: [10.48550/arXiv.2306.14048](https://doi.org/10.48550/arXiv.2306.14048).
- [49] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023, doi: [10.48550/arXiv.2309.17453](https://doi.org/10.48550/arXiv.2309.17453).
- [50] J. Xu *et al.*, “SpeContext: Enabling efficient long-context reasoning with speculative context sparsity in LLMs,” *arXiv preprint arXiv:2512.00722*, 2025, doi: [10.48550/arXiv.2512.00722](https://doi.org/10.48550/arXiv.2512.00722).
- [51] H. Zheng *et al.*, “Efficient serving for dynamic agent workflows with prediction-based KV-cache management,” *arXiv preprint arXiv:2605.06472*, 2026, doi: [10.48550/arXiv.2605.06472](https://doi.org/10.48550/arXiv.2605.06472).
- [52] Z. Bian, F. Wu, Z. Li, T. Ma, and Y. Zhuo, “TokenCake: A KV-cache-centric serving framework for LLM-based multi-agent applications,” *arXiv preprint arXiv:2510.18586*, 2025, doi: [10.48550/arXiv.2510.18586](https://doi.org/10.48550/arXiv.2510.18586).
- [53] B. Ma, J. Eitzinger, and H. Köstler, “Leyline: KV cache directives for agentic inference,” *arXiv preprint arXiv:2606.01065*, 2026, doi: [10.48550/arXiv.2606.01065](https://doi.org/10.48550/arXiv.2606.01065).
- [54] L. Stepanek, “Fail-closed lowering of resident KV claims onto LLM serving runtimes,” *arXiv preprint arXiv:2606.01387*, 2026, doi: [10.48550/arXiv.2606.01387](https://doi.org/10.48550/arXiv.2606.01387).
- [55] S. Dong *et al.*, “Memory injection attacks on LLM agents via query-only interaction,” *arXiv preprint arXiv:2503.03704*, 2025, doi: [10.48550/arXiv.2503.03704](https://doi.org/10.48550/arXiv.2503.03704).
- [56] S. S. Srivastava and H. He, “MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval,” *arXiv preprint arXiv:2512.16962*, 2025, doi: [10.48550/arXiv.2512.16962](https://doi.org/10.48550/arXiv.2512.16962).
- [57] Z. Xu, X. Zhu, Y. Yao, M. Xue, and Y. Song, “From storage to steering: Memory control flow attacks on LLM agents,” *arXiv preprint arXiv:2603.15125*, 2026, doi: [10.48550/arXiv.2603.15125](https://doi.org/10.48550/arXiv.2603.15125).
- [58] B. D. Sunil, I. Sinha, P. Maheshwari, S. Todmal, S. Malik, and S. Mishra, “Memory poisoning attack and defense on memory based LLM-agents,” *arXiv preprint arXiv:2601.05504*, 2026, doi: [10.48550/arXiv.2601.05504](https://doi.org/10.48550/arXiv.2601.05504).
- [59] Z. Lin *et al.*, “A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle,” *arXiv preprint arXiv:2604.16548*, 2026, doi: [10.48550/arXiv.2604.16548](https://doi.org/10.48550/arXiv.2604.16548).
- [60] J. Zhang *et al.*, “Beyond similarity: Trustworthy memory search for personal AI agents,” *arXiv preprint arXiv:2606.06054*, 2026, doi: [10.48550/arXiv.2606.06054](https://doi.org/10.48550/arXiv.2606.06054).
- [61] Y. Hu *et al.*, “OP-bench: Benchmarking over-personalization for memory-augmented personalized conversational agents,” *arXiv preprint arXiv:2601.13722*, 2026, doi: [10.48550/arXiv.2601.13722](https://doi.org/10.48550/arXiv.2601.13722).
- [62] H. Chao, Y. Bai, R. Sheng, T. Li, and Y. Sun, “STALE: Can LLM agents know when their memories are no longer valid?” *arXiv preprint arXiv:2605.06527*, 2026, doi: [10.48550/arXiv.2605.06527](https://doi.org/10.48550/arXiv.2605.06527).
- [63] Y. Chen, J. Zhao, B. Tang, H. Wang, F. Xiong, and Z. Li, “MemPrivacy: Privacy-preserving personalized memory management for edge-cloud agents,” *arXiv preprint arXiv:2605.09530*, 2026, doi: [10.48550/arXiv.2605.09530](https://doi.org/10.48550/arXiv.2605.09530).
- [64] Z. Chen and Q. Cheng, “Learning what to remember: A cognitively grounded multi-factor value model for agentic memory,” *arXiv preprint arXiv:2606.12945*, 2026, doi: [10.48550/arXiv.2606.12945](https://doi.org/10.48550/arXiv.2606.12945).
- [65] Q. Kang, M. Liu, S. Kai, K. Liang, T. Zhong, and M. Yuan, “Learning what to remember: Observability-safe memory retention via constrained optimization for long-horizon language agents,” *arXiv preprint arXiv:2606.10616*, 2026, doi: [10.48550/arXiv.2606.10616](https://doi.org/10.48550/arXiv.2606.10616).
- [66] P. J. Denning, “The working set model for program behavior,” *Communications of the ACM*, vol. 11, no. 5, pp. 323–333, 1968, doi: [10.1145/363095.363141](https://doi.org/10.1145/363095.363141).
- [67] L. A. Bélády, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966, doi: [10.1147/sj.52.0078](https://doi.org/10.1147/sj.52.0078).
- [68] Z. Huang *et al.*, “M3Exam: Benchmarking multimodal memory for realistic user-agent interactions,” *arXiv preprint arXiv:2606.07402*, 2026, doi: [10.48550/arXiv.2606.07402](https://doi.org/10.48550/arXiv.2606.07402).
- [69] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978, doi: [10.1145/359545.359563](https://doi.org/10.1145/359545.359563).

- [70] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: [10.1109/PROC.1975.9939](https://doi.org/10.1109/PROC.1975.9939).
- [71] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, “LongMemEval: Benchmarking chat assistants on long-term interactive memory,” in *The thirteenth international conference on learning representations*, 2025. doi: [10.48550/arXiv.2410.10813](https://doi.org/10.48550/arXiv.2410.10813).
- [72] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, “Evaluating very long-term conversational memory of LLM agents,” in *Proceedings of the 62nd annual meeting of the association for computational linguistics*, 2024, pp. 13851–13870. doi: [10.18653/v1/2024.acl-long.747](https://doi.org/10.18653/v1/2024.acl-long.747).
- [73] Y. Hu, Y. Wang, and J. McAuley, “Evaluating memory in LLM agents via incremental multi-turn interactions,” *arXiv preprint arXiv:2507.05257*, 2025, doi: [10.48550/arXiv.2507.05257](https://doi.org/10.48550/arXiv.2507.05257).
- [74] Z. He *et al.*, “MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks,” *arXiv preprint arXiv:2602.16313*, 2026, doi: [10.48550/arXiv.2602.16313](https://doi.org/10.48550/arXiv.2602.16313).
- [75] H. Tan, Z. Zhang, C. Ma, X. Chen, Q. Dai, and Z. Dong, “MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents,” *arXiv preprint arXiv:2506.21605*, 2025, doi: [10.48550/arXiv.2506.21605](https://doi.org/10.48550/arXiv.2506.21605).
- [76] W. Xu *et al.*, “MemGym: A long-horizon memory environment for LLM agents,” *arXiv preprint arXiv:2605.20833*, 2026, doi: [10.48550/arXiv.2605.20833](https://doi.org/10.48550/arXiv.2605.20833).
- [77] S. Min *et al.*, “FActScore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 12076–12100. doi: [10.18653/v1/2023.emnlp-main.741](https://doi.org/10.18653/v1/2023.emnlp-main.741).
- [78] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023, doi: [10.48550/arXiv.2309.15217](https://doi.org/10.48550/arXiv.2309.15217).